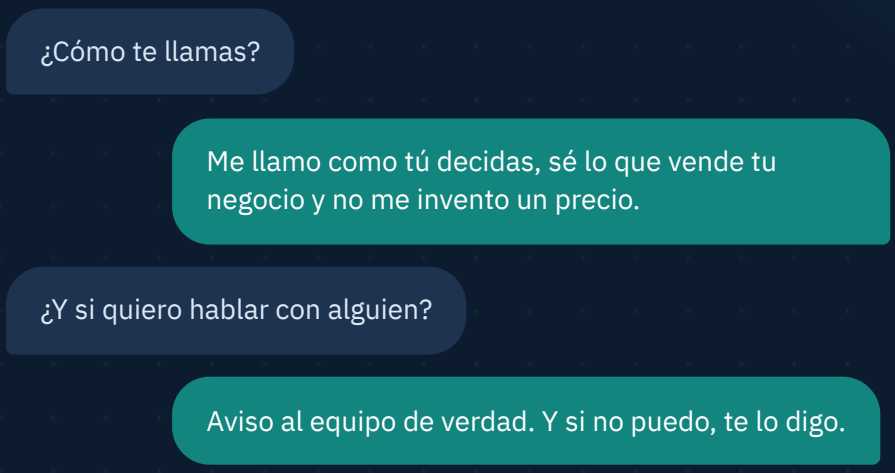
MANUAL TÉCNICO · Y PROMPTS

Un asistente para tu web

Cómo está montado Cemi, por qué cada pieza es como es, y cómo llevarlo a cualquier otro proyecto desde cero.



Parte I · Cómo funciona · Parte II · Los prompts · Anexos

Octubre de 2026

Índice

Parte 0 · Antes de empezar

0.1 Qué es este documento
0.2 Para la persona que lo va a usar
0.3 Para la IA que recibe este documento
0.4 Palabras que se usan aquí

Parte I · Cómo funciona

1. Qué es el asistente
2. El recorrido de una pregunta
3. Las piezas
4. La seguridad: lo que no se puede convencer
5. Lo que sabe: las fichas
6. Cómo contesta: los guiones
7. El guion del sistema, capa por capa
8. Lo que hace: las herramientas
9. La cadena de modelos
10. El filtro de salida
11. Las redes del servidor
12. La ventana
13. El panel del equipo
14. Costes, topes y temperatura
15. Las averías que enseñaron todo esto

Parte II · Los prompts

Cómo usar los prompts
P0 · Aprender el proyecto
P1 · Cuentas y secretos
P2 · La base de datos
P3 · Lo que sabe solo: las fichas
P4 · El cerebro: la función de servidor
P5 · La ventana
P6 · Los guiones de fábrica
P7 · Las herramientas por público
P8 · El panel del equipo
P9 · El botón que lleva a la pantalla
P10 · Ofrecerse en las páginas de producto
P11 · Las pruebas antes de encenderlo
P12 · Alimentarlo
Prompt maestro

Anexos

- Anexo A · La plantilla de un guion
- Anexo B · Los temas de fábrica
- Anexo C · El esquema de la base
- Anexo D · El código de referencia
- Anexo E · Antes de encenderlo para el público

PARTE 0

Antes de empezar

0.1 Qué es este documento

Este documento explica un asistente de chat para páginas web que ya funciona en producción —se llama **Cemi**— y trae los prompts para montar uno igual en cualquier otro proyecto, desde cero, con sus propias cuentas y accesos.

No explica el negocio donde nació Cemi. Explica **la máquina**: cómo está montado, por qué cada pieza es como es, y qué averías enseñaron cada regla. Todo lo que es propio de un negocio —qué vende, qué preguntan sus clientes, cómo habla— lo tiene que **aprender el proyecto nuevo**, y para eso están los prompts de la Parte II.

También recoge lo que enseñó **Verónica**, un asistente hermano que atendía por WhatsApp, en lo que sirve para la web. WhatsApp en sí queda fuera: aquí todo es para un sitio web.

El documento tiene tres partes:

- **Parte I · Cómo funciona.** La explicación completa, pieza por pieza.
- **Parte II · Los prompts.** Para pegarlos, en orden, en el proyecto nuevo.
- **Anexos.** Plantillas, el esquema de la base y el código de referencia.

0.2 Para la persona que lo va a usar

Lo que necesitas tener antes de empezar:

1. **Un proyecto de Supabase** para el proyecto nuevo. Da la base de datos, las cuentas de usuario y las funciones de servidor. Si el proyecto ya usa Supabase, sirve el mismo.
2. **Una clave de un proveedor de modelos de IA** compatible con la API de chat de OpenAI: Groq, OpenAI, OpenRouter, Together, Mistral, DeepSeek… Mejor dos de proveedores distintos (ver §9).
3. **Acceso al código del sitio** y una IA que trabaje sobre él (por ejemplo, Claude Code abierto en ese repositorio).
4. **Saber quién atiende de verdad.** El asistente avisa al equipo cuando hace falta una persona: tiene que haber alguien a quien avisar, y un canal (correo, notificaciones del panel) por donde avisarle.

Cómo se usa:

1. Dale este documento entero a la IA del proyecto nuevo (el PDF, o su texto).
2. Pega el **Prompt 0**. La IA estudiará el proyecto y te devolverá un informe con una propuesta. **No escribe código todavía.**
3. Revisa la propuesta, corrige lo que no sea así y responde sus preguntas.

4. Sigue con los prompts 1 a 12, en orden, uno por uno. Cada uno dice qué entrega y cómo comprobarlo.
5. Si prefieres no ir paso a paso, está el **Prompt maestro**, que lo hace todo por fases y se para a enseñarte cada una.

Los secretos no se pegan en el chat. Las claves del proveedor de IA se ponen en el panel de Supabase (Edge Functions → Secrets). La IA puede comprobar que existen sin verlas.

0.3 Para la IA que recibe este documento

Si eres la IA que va a montar el asistente, estas son tus reglas:

- **Lee el documento entero antes de tocar nada.** Las piezas dependen unas de otras y casi todas las reglas existen por una avería concreta (§15).
- **Primero estudia el proyecto** (Prompt 0). El asistente tiene que hablar del negocio de **este** proyecto, con sus datos reales. No copies nombres, textos, precios ni preguntas de Cemi: no existen aquí.
- **Lo que solo sabe el dueño, pregúntalo.** El nombre y la personalidad del asistente, a quién se avisa, qué puede prometer y qué no. No lo inventes.
- **Adapta los nombres a la casa.** Aquí las tablas se llaman `bot_*`. Si el proyecto tiene sus propias convenciones (prefijos, idioma, estilo de comentarios), síguelas.
- **Nunca pongas un secreto en el código** ni en un archivo del repositorio.
- **No digas que algo funciona sin haberlo probado.** La Parte II trae cómo comprobar cada paso; si algo no se pudo probar, dilo.
- **Si el proyecto no usa Supabase,** cada sección trae una nota de cómo traducir la pieza. La idea se mantiene; cambia la herramienta.

0.4 Palabras que se usan aquí

Palabra	Qué significa
Ámbito	A quién atiende el asistente en esa conversación: `visitante` (sin cuenta), `usuario` (con cuenta) o `equipo`. Lo decide la base, no la ventana.
Ficha	Un dato que el asistente sabe: un producto con su precio, cómo se paga, un horario. Unas las genera la base sola; otras las escribe el equipo.
Guion	Una conversación de ejemplo escrita por el equipo: lo que dice la persona y dos o tres formas de contestar. Enseña el **tono**, no los datos.
Herramienta	Algo que el asistente puede **hacer**, no solo contar: consultar un pedido, guardar un contacto, avisar al equipo. Cada una es una función de la base.
Cadena de modelos	La lista de modelos de IA que se prueban en orden. Si uno falla, se usa el siguiente.
Degradado	Cuando todos los modelos fallaron y el asistente contestó con una frase de cortesía. Se marca y se ve en la pantalla.
Escalar	Avisar al equipo de que una conversación necesita una persona.

Palabra	Qué significa
Huella	Un identificador al azar que guarda el navegador de un visitante. Separa conversaciones; **no identifica a nadie**.
Guion del sistema	El texto que recibe el modelo antes de cada respuesta: quién es, qué sabe, a quién atiende y cómo contesta.

PARTE I

Cómo funciona

1. Qué es el asistente

Es una burbuja de chat que aparece en todas las páginas del sitio. Tiene **nombre y personalidad**: no es «el chat de soporte», es alguien con cara (una mascota o un dibujo) que contesta como lo haría una persona del equipo.

Atiende a **tres públicos con el mismo motor**:

Público	Quién es	Qué puede saber de él
Visitante	Alguien que mira la web sin haber entrado a una cuenta.	Nada. Solo lo que es público: el catálogo, los precios, cómo se compra.
Usuario	Un cliente con cuenta abierta (alumno, comprador, paciente…).	Lo suyo y nada más: sus pedidos, sus pagos, su avance.
Equipo	Alguien que trabaja en el negocio, con su rol.	Lo que su rol le deja ver en la plataforma, ni una fila más.

Lo único que cambia entre uno y otro es **lo que pueden ver**, y eso no lo decide una frase del guion: lo decide la base de datos (§4).

Vive en una función de servidor, no en un ordenador de nadie. Es la primera decisión y condiciona todas las demás. Verónica, el asistente de WhatsApp, corría en un portátil: cuando se cerraba la tapa, el negocio dejaba de contestar. En otro bot del mismo tipo se midieron 312 reconexiones en un día y una caída de 1 h 46 min en plena hora de venta. Una función de servidor no se duerme, no se queda sin batería y no depende de que alguien deje un equipo encendido.

Qué hace, en resumen:

- **Cuenta.** Responde con los datos reales del negocio, que lee de la base en cada pregunta. Si un dato no está, no lo inventa.
- **Hace.** Puede ejecutar acciones con herramientas: guardar los datos de un interesado, consultar un pedido, avisar al equipo de verdad.
- **Aprende.** No reentrenando el modelo, sino con lo que el equipo le enseña: fichas (datos) y guiones (cómo contestar). Lo que se le enseña hoy lo usa en la siguiente respuesta, y lo que se apaga deja de influir en el acto.
- **Se deja vigilar.** Guarda cada conversación, cada error y lo que hizo, y el equipo lo ve todo en un panel.

2. El recorrido de una pregunta

Esto es lo que pasa, en orden, cuando alguien escribe en la burbuja:

1. **La ventana manda la pregunta** a la función de servidor `asistente`, con el ámbito que cree que le toca y el número de la conversación. Si es un visitante, manda además su huella.
2. **La función mira quién pregunta.** Si la petición trae la sesión de un usuario, todo lo demás se hará **con su token**. Si es un visitante, se va por un camino aparte (§4).
3. **Comprueba los topes.** Para el visitante: cuántos mensajes lleva en la última hora, y cuántos hay en total. Si se pasó, contesta una frase amable que ofrece dejar sus datos, sin llamar al modelo.
4. **Pide el contexto a la base.** Una función devuelve, en un solo JSON, todo lo que esa persona puede saber: el catálogo, las fichas, lo suyo y, si es del equipo, las cifras de su rol. La base decide el ámbito real: pedir «equipo» no te hace del equipo.
5. **Lee el hilo:** los últimos 24 mensajes de la conversación (16 para un visitante), para que no vuelva a preguntar lo que ya le dijeron.
6. **Elige los guiones.** La base busca los que se parecen a lo que acaba de escribir la persona y a sus dos mensajes anteriores, y devuelve los tres mejores (§6).
7. **Arma el guion del sistema,** por capas: el oficio (quién es y cómo escribe), cómo usar las herramientas, el encargo según el público, los datos, lo de quien pregunta y los ejemplos (§7).
8. **Llama al modelo** por la cadena de modelos. Si el primero falla, prueba el siguiente (§9).
9. **Si el modelo pide herramientas, las ejecuta** con el token de quien pregunta, le devuelve los resultados y vuelve a preguntarle. Como mucho tres vueltas; la última, sin herramientas, para obligarle a contestar (§8).
10. **Limpia la respuesta** con el filtro de salida: lo que el guion prohíbe y se puede detectar por texto se quita también en código (§10).
11. **Aplica las redes del servidor.** Si dijo que avisó al equipo y no lo hizo, avisa el servidor. Si volvió a pedir datos que ya pidió, se le quita la frase (§11).
12. **Guarda el turno** —pregunta, respuesta, modelo, tokens, tiempo, error— y solo entonces responde. Si la respuesta habla de una pantalla del sitio a la que esa persona puede entrar, añade un botón que lleva allí (§12).

> **Nota si no usas Supabase.** El recorrido es el mismo en cualquier backend. Lo que hay que conservar es el orden y una regla: los pasos 4 y 9 se hacen **con la identidad de quien pregunta**, nunca con una cuenta de servicio que lo ve todo.

3. Las piezas

Pieza	Dónde vive	Qué hace
`asistente`	Función de servidor	El punto de entrada. Recibe la pregunta, decide el camino y devuelve la respuesta.

Pieza	Dónde vive	Qué hace
`cerebro.ts`	Módulo compartido	La cadena de modelos, el bucle de herramientas y el filtro de salida.
`herramientas.ts`	Módulo compartido	El catálogo de lo que el asistente puede hacer y a quién se le ofrece.
`pantallas.ts`	Módulo compartido	La lista de pantallas del sitio, para el botón «Ir a…».
`bot_contexto(ámbito)`	Base, **security invoker**	Lo que puede saber quien pregunta. Corre con sus permisos.
`bot_contexto_publico()`	Base, security definer	Lo público, para el visitante.
`bot_conocimiento`	Tabla	Las fichas: las automáticas y las del equipo.
`bot_refrescar()`	Base	Regenera las fichas automáticas desde los datos del negocio.
`bot_guiones`	Tabla	Los guiones, con sus disparadores y cuántas veces se usaron.
`bot_guiones_para(texto, ámbito)`	Base	Elige los guiones que se parecen a la pregunta.
`bot_conversaciones` / `bot_mensajes`	Tablas	La memoria y el registro de todo lo dicho.
`bot_escuchado`	Tabla	Lo que pregunta la gente, agrupado, para saber qué enseñarle.
`bot_borradores`	Tabla	Lo que el asistente **prepara** y una persona confirma.
`bot_escalar(conversación, motivo)`	Base	Avisa al equipo, de verdad.
`bot_visitante_permitir(huella)`	Base	Los topes de gasto del visitante.
`bot_dejar_contacto(...)`	Base	Guarda los datos de un interesado.
`asistente.js`	Navegador	La burbuja, la ventana y el hilo de la conversación.
Panel del asistente	Página del equipo	Lo que sabe, cómo contesta, conversaciones, salud y ajustes.

Todas las tablas tienen RLS encendido y cero políticas. No se llega a ellas más que por las funciones, y cada función comprueba primero quién llama. Así un navegador no puede leer ni escribir nada del asistente por su cuenta.

4. La seguridad: lo que no se puede convencer

Esta es la parte que más importa, y conviene entender por qué está montada así.

La forma fácil está mal. Habría bastado con escribir en el guion «no des datos de otras personas». Pero eso es defender el sistema con algo que **se puede convencer**: al modelo se le insiste, se le da lástima, se le dice que es una emergencia, y a la cuarta vez lo suelta. Es un fallo documentado, no una hipótesis.

La forma que funciona: los permisos viven en la base.

- La función de servidor **nunca usa la llave de servicio para leer datos de personas.** Toma el token de quien pregunta y llama al contexto y a cada herramienta **con él**.
- `bot_contexto` y las herramientas de datos están declaradas **sin** `security definer` (son `security invoker`): corren con los permisos de quien las llama, así que las reglas de fila se aplican igual que si esa persona consultara a mano.
- Un usuario que pida datos de otro recibe **cero filas antes de que el modelo vea nada.** No hay nada que convencer.
- El ámbito lo decide la base. Si un usuario manda `ámbito: 'equipo'` desde la consola del navegador, la función le devuelve el ámbito que le corresponde, no el que pidió.

El visitante va por un camino aparte, a propósito. No tiene token, así que no se puede aflojar el camino normal para él: se le atiende con la llave de servicio, y por eso **solo** se le deja tocar lo que ya es público (el catálogo) y una sola herramienta: dejar sus datos de contacto. Ese camino se decide antes de leer ningún contexto, para que nunca se llegue a pedir datos de una persona que no existe.

«La carta no es la cerradura». El archivo `herramientas.ts` dice a quién se le **ofrece** cada herramienta: un usuario ve las suyas, alguien de cobros ve las de cobros. Pero esa lista no es el sistema de permisos, es la carta del restaurante: evita que el modelo se confunda con herramientas que no le tocan. Si se le colara una, la base no le daría las filas. Lo mismo con los botones «Ir a…»: la pantalla vuelve a comprobar el rol al abrirse.

Los parámetros que deciden identidad los pone el servidor. Cuando el modelo pide «avisar al equipo», no elige **qué** conversación se escala: el servidor la añade (`delServidor`) y el modelo ni la ve ni la puede escribir. Y solo pasan a la base los parámetros declarados en la herramienta: si el modelo se inventa un `p_usuario_id` , se cae antes de llegar.

Una huella no es una identidad. La huella del visitante la escribe su navegador y cualquiera la puede cambiar. Sirve para separar conversaciones, nunca para reconocer a nadie. Es la misma lección que dejó Verónica por WhatsApp: allí lo único que llegaba era un número, y **un número no es una contraseña**. En la web, lo único que identifica a alguien es su sesión.

> **Nota si no usas Supabase.** «Security invoker» significa: la consulta se hace con la identidad del usuario, y la base (o tu capa de datos) filtra las filas por esa identidad. Con otro backend, haz que cada herramienta reciba el usuario autenticado de la petición —nunca del modelo— y que todas sus consultas lleven el filtro por ese usuario. Nunca un «modo administrador» para el asistente.

---

5. Lo que sabe: las fichas

El asistente no se aprende nada de memoria. **En cada pregunta** recibe lo que sabe en ese momento, leído de la base. Por eso, si cambia un precio, la siguiente respuesta ya usa el nuevo.

Hay dos clases de fichas en la tabla `bot_conocimiento` :

1. **Las automáticas** (`origen = 'plataforma'`). Las genera `bot_refrescar()` a partir de los datos del negocio: una por cada producto o servicio publicado (nombre, de qué va, precio, duración, partes), una con las formas de pago activas, otra con las fechas abiertas. Cada una lleva una `clave` única (`producto:<id>` , `pagos` …) para poder reescribirla.
2. **Las del equipo** (`origen = 'equipo'`). Las escribe una persona en el panel: horarios de atención, políticas, respuestas a lo que la gente pregunta mucho.

Se entera sola de los cambios. Hay disparadores en las tablas del catálogo: al publicar un producto, cambiar un precio o abrir una fecha, las fichas se regeneran en el acto. Dos detalles que no son detalles:

- **Van `for each statement` , no por fila.** Publicar diez productos de golpe regenera una vez, no diez.
- **Van envueltos en su propio manejador de errores.** Si la ficha no se puede regenerar, el producto se publica igual. Que el asistente se quede con el catálogo de ayer es un problema pequeño; que no se pueda publicar por culpa del asistente sería grande, y de los que nadie relaciona con su causa.

Y hay un botón **«Actualizar lo que sabe»** en el panel, por si acaso.

Lo que se escribe a mano no se pisa. Si alguien edita una ficha automática para afinarla, pasa a ser «del equipo» y los disparadores dejan de tocarla. Si no, el siguiente cambio de precio se llevaría por delante un texto que alguien afinó a propósito. El panel avisa antes de editar una automática.

Las fichas de lo que ya no existe se apagan, no se borran. Si un producto deja de estar publicado, su ficha queda inactiva.

Un dato ausente se declara, no se omite. Es la regla más importante de esta sección. Si el catálogo llega vacío porque no se pudo leer, el guion no se queda corto: dice explícitamente «no tienes el catálogo, tienes prohibido decir precios» y le da la frase de espera ya escrita. En un bot real, con el catálogo vacío por un servidor lento, el modelo se inventó precios **un 85 % por encima** del real, con total naturalidad.

Y hay que distinguir dos vacíos que no son el mismo:

Situación	Lo que se le dice al modelo
El catálogo **no se pudo leer** (fallo técnico).	«No tienes el catálogo. No lo menciones. Tienes prohibido decir precios o fechas. Si preguntan, responde: “Déjame confirmarte eso con el equipo y te escribo”.»
El catálogo **se leyó y está vacío** (no hay nada a la venta ahora).	«Ahora mismo no hay nada abierto. No es un fallo. Dilo sin rodeos, no des precios ni fechas, y ofrece avisar cuando abra.»

Tratarlos igual llevó a Cemi a pedir nombre y correo «para mandar los precios» tres veces seguidas, cuando lo honesto era decir que no había nada abierto.

«Lo que preguntan». La tabla `bot_escuchado` guarda lo que pregunta la gente (sin datos personales: correos y teléfonos se tachan antes de guardar, y se ignoran los mensajes de menos de 12 caracteres, como «ok» o «gracias»). El panel las agrupa por pregunta parecida y las ordena por cuántas veces salieron. Al lado de cada grupo hay dos botones: **«Enseñarle el dato»** (crea una ficha) y **«Enseñarle cómo contestar»** (crea un guion). Es la forma más directa de saber qué le falta.

> En Cemi esta lista la alimentaba el canal de WhatsApp. En un proyecto web, alimenta `bot_escuchado` desde la función `asistente` con cada pregunta de visitantes y usuarios.

6. Cómo contesta: los guiones

Un asistente con **datos** y sin **ejemplos** contesta como una base de datos: la misma frase seca, y a la segunda la repite. Le pasó a Cemi con un visitante real:

> «¿cómo te llamas?» → «Soy del equipo de [negocio].» «¿pero cómo te llamas?» → lo mismo, palabra por palabra. «¿no tienes nombre?» → «No, no tengo nombre.»

Lo que le faltaba no eran datos: eran ejemplos de cómo se le habla a una persona. Para eso están los guiones.

Qué es un guion. Una conversación de ejemplo, en texto plano:

```
TEXTO

Persona: como te llamas?
[ASISTENTE]: Me llamo [ASISTENTE]. Soy quien te atiende aquí en la web…
[ASISTENTE]: [ASISTENTE], para servirte. ¿En qué te ayudo?
Si dice «eres un bot?» → [ASISTENTE]: Soy [ASISTENTE], el asistente de
  [NEGOCIO]. Lo que te digo sale de lo que el equipo me enseñó…
```

- Cada `Persona:` es lo que escribe la gente.
- Cada línea del asistente es **una forma** de contestar. Van dos o tres, y por eso no responde siempre igual: elige, mezcla y cambia las palabras.
- `Si dice «…» → …` es una rama: lo que contesta si la persona sigue por ahí.
- Lo que va entre corchetes `[así]` es un **hueco** que rellena con el dato real de las fichas. **Ningún guion lleva precios ni fechas escritos**: eso sale de la base y solo de la base. Si no tiene el dato, no lo dice.
- `[usa dejar_contacto]` o `[usa avisar_al_equipo]` dentro de un ejemplo le dice que en ese punto **llame a la herramienta**: la frase sola no vale.

Cómo se eligen. Cada guion tiene un **ámbito** (visitante, usuario, equipo o todos), un **tema** y sus **disparadores**: palabras o frases que, si aparecen, lo traen. Antes de cada respuesta, `bot_guiones_para` puntúa los guiones activos de ese ámbito contra la pregunta de ahora **y las dos anteriores** de la persona (porque «¿y en cuotas?» solo se entiende con el «¿cuánto cuesta?» de antes):

- **3 puntos** por cada disparador que aparezca entero.
- **1 punto** por cada palabra de 4 letras o más que coincida con el tema o los disparadores.
- Hacen falta **2 puntos como mínimo.** Una sola palabra suelta («como») traía guiones de más.
- Se llevan los **3 mejores**. Si dos empatan, primero el que menos se ha usado, así con el tiempo se reparten.
- Además, los marcados **«siempre»** entran en todas las respuestas. Debería haber uno solo: cómo salir de lo que no toca (chistes, tareas escolares, insultos), que no tiene palabras con las que dispararlo.

¿Por qué tres y no más? Por peso. Un turno pesa unos 2.000 tokens (oficio, catálogo, guiones, hilo), y los planes gratuitos de algunos proveedores dan 8.000 tokens por minuto y por modelo: cuatro turnos por minuto. Cada guion de más es menos gente atendida por minuto (ver §14).

Cómo llegan al modelo. Como un bloque «ASÍ CONTESTAS» con sus reglas:

- No los copies letra por letra: son el tono y el fondo.
- Elige entre las variantes, mezcla, acorta.
- Nunca la misma frase dos veces en una conversación.
- Los corchetes se rellenan con el dato real o se callan.
- Cuando un ejemplo diga `[usa …]`, llama a esa herramienta de verdad.

Si en el panel se cambia el nombre del asistente, el servidor reemplaza el nombre en los guiones antes de dárselos al modelo, para que no aprenda dos nombres.

Cómo se le enseña. En el panel, pestaña **«Cómo contesta»**:

- La lista de guiones, con cuántas veces se usó cada uno.
- Un editor con el formato explicado al lado.
- **Un probador**: escribes lo que escribiría alguien y ves qué guiones usaría, **sin gastar un turno del modelo**. Si no usa ninguno, esa pregunta merece un guion.
- En **«Conversaciones»**, cada respuesta real tiene dos botones: **«Así se contesta»** la guarda como ejemplo; **«Corregir»** abre el editor con la pregunta puesta para escribir cómo debería haber contestado. Las dos quedan marcadas «de una conversación real».

Lo que NO es. No es entrenar el modelo. El modelo es el mismo; lo que cambia es lo que se le enseña antes de cada respuesta. Es la forma honesta de que «aprenda» sin dejar de poder corregirlo: un guion malo se apaga y deja de influir en el acto.

Los de fábrica. El proyecto nuevo arranca con 20–30 guiones escritos para su negocio (Anexo B trae la lista de temas). Uno de fábrica que el equipo edita pasa a ser del equipo, y la siguiente siembra no lo pisa.

---

7. El guion del sistema, capa por capa

Es el texto que recibe el modelo antes de cada respuesta. Se arma en el servidor, en este orden:

1. El oficio —lo que no cambia entre públicos—:

- Su **nombre**, que sale de la configuración y no del código. Sin esta línea, Cemi contestaba «no tengo nombre».
- Quién es, de qué negocio, y que tiene personalidad.
- Idioma y trato.
- **Cómo escribe:**
  - breve, una o dos frases;
  - texto plano, sin markdown;
  - casi sin emojis, y nunca el mismo dos veces seguidas;
  - nunca repite palabra por palabra algo que ya dijo;
  - cierra casi siempre con una pregunta corta o una puerta abierta.
- **Lo que no hace nunca:** inventar precios, fechas ni datos de pago; confirmar que un pago entró; prometer reembolsos, descuentos, excepciones ni plazos que no pueda comprobar.

- No dice que es un bot ni una IA. Si insisten, no lo discute: dice quién es, sigue ayudando y ofrece que le escriba una persona.

2. Cómo usar las herramientas. Sin esto, el modelo falla en dos direcciones opuestas: promete acciones que no ejecuta, o llama herramientas para saludar. Las reglas:

- Cuando alguien pide algo que una herramienta hace, llámala. No describas lo que vas a hacer: hazlo y cuenta el resultado.
- Nunca digas que avisaste al equipo sin llamar a `avisar_al_equipo`.
- Si piden hablar con una persona, o dudan de que lo seas, no lo discutas: avisa al equipo y di que ya avisaste. Esta regla gana sobre las demás.
- Para saludar, charlar o dar las gracias, ninguna herramienta.
- Si una herramienta devuelve varios candidatos, no elijas tú: pregunta cuál.
- Si devuelve un error, dilo con palabras normales; no lo escondas ni lo inventes.
- Las que **preparan** algo no lo mandan: di a cuántos iría y que falta que alguien lo confirme.

3. El encargo, según el público:

- **Visitante:** no tiene cuenta y no sabes nada de él; no le pidas datos «para buscarlos», no hay nada que buscar. Tu trabajo es uno: que entienda si algo le sirve. Si le interesa, pide nombre y correo **una sola vez** y usa `dejar_contacto`. Si pregunta por los datos de una persona: «Eso lo ve cada quien entrando a su cuenta».
- **Usuario:** ayúdale con lo suyo; cuando le mandes a una pantalla, nómbrala como se llama en el sitio. De otras personas no tienes nada.
- **Equipo:** el oficio de su rol en una o dos líneas (qué lleva, qué pantallas usa, qué no es de su trabajo y a quién mandarlo). Puedes hablar de cifras; un número inventado en una decisión de negocio cuesta más que no tenerlo.

> El oficio de cada rol **no da ni quita permisos**: eso ya lo decidió la base. Solo cambia de qué se le habla, para que no le conteste a quien cobra cosas de quien da clase.

4. Los datos: el catálogo (o el bloque que le prohíbe hablar de él, §5), el negocio, cómo se paga, la próxima fecha, los contactos públicos que puede dar, y las fichas del equipo.

5. Lo de quien pregunta (solo usuario y equipo): su nombre, cómo llamarle, y lo suyo como **hechos que puede usar**, no como una ficha para recitar. Si no se inyecta, vuelve a preguntarlo; si no se le dice cómo usarlo, lo lee en voz alta y suena a base de datos. Para el equipo, además, las cifras de su rol.

6. Los ejemplos: el bloque «ASÍ CONTESTAS» con los guiones elegidos (§6).

Lo último que lee pesa más. Cuando una regla es crítica para esa conversación —por ejemplo, «ya le pediste sus datos, no los pidas otra vez»—, se pone en el encargo **y** se repite al final del guion.

---

8. Lo que hace: las herramientas

Al principio Cemi solo **contaba**: se le metía un resumen y hablaba de él. A «¿puedes avisar al equipo?» contestaba que sí, y no avisaba a nadie, porque no tenía con qué. Las herramientas son el con qué.

Cada herramienta es una función de la base. Las de datos son `security invoker` y se ejecutan con el token de quien pregunta, así que **no hay un solo control de permisos escrito a mano**: si a alguien no le toca algo, la base no le da las filas.

Tres niveles, y el nivel dice lo que cuesta:

Nivel	Qué hace	Ejemplo genérico
Cuenta	Solo lee. La seguridad ya está resuelta.	«¿Dónde está mi pedido?», «¿Cuánto debo?»
Prepara	Redacta la acción y la guarda en `bot_borradores`. Una persona la confirma en el panel («Por confirmar»).	«Manda un recordatorio a los que no han pagado.»
Hace	Escribe en la base. Queda anotada en auditoría y se puede deshacer.	«Apúntame a esa fecha.» «Guarda que me avisen tres días antes.»

Lo que manda mensajes a mucha gente o mueve dinero va en **Prepara**, nunca en **Hace**.

Cómo se declara una herramienta (en `herramientas.ts`):

Campo	Para qué
`nombre`	El nombre que ve el modelo. Un verbo claro: `donde_esta_mi_pedido`.
`rpc`	La función de la base que la ejecuta.
`descripcion`	Cuándo usarla y qué **no** hace. «No confirma que un pago entró.»
`parametros`	Los que el modelo puede rellenar, con su tipo y una frase.
`obligatorios`	Los que no pueden faltar.
`delServidor`	Los que pone el servidor y el modelo no ve (la conversación, por ejemplo).
`ambito` / `roles`	A quién se le **ofrece**. La carta, no la cerradura.
`escribe` / `entidad` / `idEnRespuesta`	Si escribe: qué tabla toca y dónde viene el id, para anotarlo y poder deshacer.

El bucle (en `cerebro.ts`):

- Como mucho **4 llamadas** por vuelta y **3 vueltas**.
- **La última vuelta va sin herramientas.** Si no, un modelo atascado pidiendo la misma llamada sale del bucle sin haber escrito nada, y la persona no ve respuesta.
- Un nombre que no está en la carta **no se llama**: los modelos se inventan herramientas que les vendrían bien.
- Argumentos que no se pueden leer → error al modelo, no a la base.
- **Solo pasan los parámetros declarados.** Los que pone el servidor se añaden después.
- **El error se le devuelve al modelo tal cual.** Un «ese pedido no existe» tiene que llegarle para que lo diga; si se le oculta, se inventa que salió bien.
- **Lo que escribe deja rastro en un solo sitio** (`bot_anotar_accion`), no repartido por cada función: una herramienta nueva lo hereda sin que nadie se acuerde de añadirlo.

Las dos herramientas que todo proyecto necesita:

- `avisar_al_equipo` → `bot_escalar` (ver §11). No se ofrece si no hay una conversación abierta: no habría qué escalar, y ofrecerla llevaría a prometer un aviso imposible.
- `dejar_contacto` → `bot_dejar_contacto`: la única del visitante. Guarda nombre, correo, teléfono y lo que le interesa en la tabla de contactos del proyecto. Si el correo ya existe, **suma** el mensaje al contacto en vez de fallar (la persona más interesada es la que vuelve a escribir), y tiene su propio tope.

---

9. La cadena de modelos

Los modelos van en un secreto, no en el código. Los proveedores retiran modelos sin avisar. A Cemi le pasó: los dos modelos de su cadena se apagaron el mismo día y el asistente se quedó mudo sin que cambiara una línea de código. Con la lista en un secreto, se arregla cambiando el secreto, **sin volver a desplegar**.

```
TEXTO
ASISTENTE_MODELOS = proveedorA:modelo-grande,proveedorA:modelo-pequeño,proveedorB:otro-modelo
```

Dalo por caducado desde el primer día. Cuando empiece a contestar la frase de cortesía sin motivo, mira en el panel si el error dice `model_not_found` y cambia el secreto.

Varios eslabones, y mejor de proveedores distintos. Los dos eslabones de Cemi eran de la misma casa y cayeron juntos. Una cadena de respaldo de verdad mezcla proveedores. Con la API compatible con OpenAI, cada proveedor es una URL y una clave:

Proveedor	Dirección	Secreto
Groq	`https://api.groq.com/openai/v1/chat/completions`	`GROQ_API_KEY`
OpenAI	`https://api.openai.com/v1/chat/completions`	`OPENAI_API_KEY`
OpenRouter	`https://openrouter.ai/api/v1/chat/completions`	`OPENROUTER_API_KEY`

Los 429 (demasiadas peticiones) no son una caída. Cada modelo tiene su propio contador por minuto, así que ante un 429:

1. Se anota cuánto pide esperar (cabecera `retry-after` o el texto «try again in 1.2s»).
2. Se pasa **ya** al siguiente eslabón, sin esperar: casi siempre otro está libre.
3. Si **todos** dijeron 429 y el que antes se libera pide 6 segundos o menos, se espera eso y se recorre la cadena **una vez más**. Nunca más de una.

Con un plan gratuito pequeño, una prueba de diez preguntas seguidas dejó a Cemi diciendo la frase de avería. Lo que lo arregla de verdad es un plan de pago con más límite; la cadena larga solo amortigua.

El esfuerzo de razonamiento. Los modelos que razonan pueden gastar todo el presupuesto de respuesta razonando y devolver **contenido vacío sin lanzar error**: el bot parece colgado. Hay que pedirles poco esfuerzo, pero cada familia usa su palabra (`reasoning_effort: "low"` en unos, `"none"` en otros), y mandar la de otra familia **tumba la petición entera**. Así que:

- Se decide por familia de modelo, y al que no razona no se le manda nada.
- Si el modelo responde 400 quejándose del parámetro, se repite la petición sin él, en vez de darlo por caído.

Vacío es fallo solo si tampoco pidió herramientas. Cuando el modelo decide usar una, el texto viene vacío a propósito.

El error de la cadena dice qué arreglar. Si se agotan todos, el error arrastra el motivo de **cada** eslabón. «Ningún modelo respondió» a secas no distingue entre una clave que falta, una clave mal pegada y un modelo retirado: tres arreglos distintos.

Diagnóstico de la clave sin enseñarla. Si falta el secreto, el error dice los nombres de secretos **parecidos** que sí existen (nunca sus valores): no es lo mismo no haberla puesto que haberla puesto como `GROQ_KEY`. Y si la clave no empieza por el prefijo que usa ese proveedor, lo dice: suele ser que se pegó con espacios o a medias.

La cadena nunca termina en silencio (ver §11).

---

10. El filtro de salida

El guion es una preferencia; el filtro es la garantía. Todo lo que se prohíbe y se puede detectar por texto se limpia también en código antes de enviar:

- **Bytes de control.** Algunos modelos dejan escapar un byte suelto de sus marcas internas, que en la burbuja sale como un cuadradito.
- **Markdown.** La burbuja escapa el HTML (hace bien), así que unos `**` salen en crudo. Se quitan negritas, cursivas, código y títulos.
- **Signos de apertura** (`¿`, `¡`), si el estilo de la casa no los usa. Es un ejemplo de regla de estilo que se garantiza en código.
- **Las frases que delatan al modelo** («como IA», «soy un modelo de lenguaje»…) se quitan **enteras**, partiendo el texto por frases. No se cortan «hasta el siguiente punto»: cuando no había punto donde el filtro suponía, se comía el centro y un cliente recibió una ristra de puntos suspensivos.

Y el filtro NUNCA puede dejar la respuesta vacía. Si al limpiar no queda nada, gana el texto original. Salió de una avería real: la lista de frases prohibidas incluía «no tengo acceso a», y a «ya pagué, ¿me lo confirmas?» el asistente contestó «No tengo acceso a esa confirmación» —una respuesta honesta y correcta— que el filtro se comió entera. La pantalla enseñó una avería que no existía. **La lista de frases prohibidas debe ser corta.**

`pareceTexto`. Una comprobación aparte de que lo que va a salir es texto humano: más de 8 caracteres, al menos la mitad letras, y sin ristras de puntos. No juzga si la respuesta es buena; solo si es texto.

---

11. Las redes del servidor

Un modelo no es un sitio donde poner una garantía. Cada promesa que el asistente hace a una persona la comprueba el servidor.

La promesa de «aviso al equipo». Costó tres intentos que fuera verdad, y las tres capas parecen redundantes pero ninguna lo es:

1. **Que la función haga algo.** `bot_escalar` notifica a las personas que correspondan (en el panel y por correo), con enlace a la conversación, y no repite el aviso dentro de 6 horas. Si el motivo habla de dinero, avisa además a quien lleva los cobros.
2. **Que alguien la llame.** Es la herramienta `avisar_al_equipo`, y el guion le dice cuándo usarla.
3. **Que el servidor lo compruebe.** Probándolo, el modelo llamó a la herramienta, **la llamada falló**, se le devolvió el error, y aun así contestó «Ya avisé al equipo, pronto te contactarán». Así que, después de cada respuesta, el servidor mira si el texto **dice** que se avisó («avisé», «te escribirán», «te contactarán»…). Si lo dice y no está avisado, escala él mismo, con la conversación que conoce el servidor. Y si tampoco puede, cambia la frase por una que no promete nada.

Se avisa de más a propósito. Un aviso sobrante le cuesta al equipo mirar una conversación; uno que falta le cuesta a alguien a quien le dijeron que le iban a escribir y nadie le escribió.

Pedir los datos una sola vez. «Pídele nombre y correo una sola vez» estaba en el guion, y el modelo los pidió tres veces seguidas: a «¿cuánto cuesta?», a «¿en cuotas?» y hasta a «ok, gracias». Ahora:

1. El servidor mira el hilo: si el asistente **ya** los pidió, el encargo cambia a «ya se los pediste, no los vuelvas a pedir», y se repite al final.
2. A los guiones elegidos se les quitan las líneas que piden el contacto, para que los ejemplos no le empujen.
3. Si aun así la respuesta los pide, se le **quita esa frase**.
4. Si la respuesta **entera** era pedirlos, se le da una vuelta más al modelo con la negativa dicha como la diría la persona («No te voy a dar mis datos ahora. Contesta solo lo que te pregunté»), sin herramientas y con menos temperatura. Si tampoco, una frase de la casa que no pide nada.

Ojo: reconocer «pedir datos» exige una **petición** (con pregunta o con verbo de pedir). «Ya guardé tu nombre y tu correo» no lo es.

Nunca termina en silencio, y el fallo se ve. Si se agotan todos los modelos, se contesta una frase de cortesía honesta y la respuesta se marca **degradada**. La ventana enseña un aviso pequeño («No pude consultar bien ahora mismo. Ya avisé al equipo»). Sin esa marca, la única señal de que el asistente está caído sería que empieza a contestar raro, y eso se descubre tarde y por un cliente. Cuando se dice «ya avisé al equipo», **se avisa**: si no, la mentira sería nuestra.

Las frases de fallo son distintas para cada público. Al visitante no se le puede «avisar al equipo» —no hay de parte de quién—, así que la salida honesta es: «Ahora no te puedo responder bien por aquí. Si me dejas tu nombre y tu correo, te escribe una persona del equipo».

Se guarda después de responder, nunca antes. Confirmar antes de completar convierte cualquier caída en pérdida definitiva.

El error de guardar no tumba la respuesta. Si no se pudo guardar el turno, se escribe en el log y la persona recibe su respuesta igual.

Los errores que se tragan, se escriben. La memoria del visitante falló en silencio durante semanas: la tabla no admitía el canal «visitante», el insert se rechazaba, la función se tragaba el error, y cada mensaje se contestaba como si fuera el primero (de ahí el «soy del equipo» repetido). Ahora todo error que no se puede

resolver se escribe en el log.

No se borra nada. No hay botón de borrar conversaciones: lo que dijo el asistente es la única prueba de lo que dijo el asistente.

El hilo no se recorta para ahorrar. Para eso está el tope de la pregunta. Recortar el hilo degrada la conversación en silencio: el asistente vuelve a preguntar lo que le acaban de decir.

---

12. La ventana

`asistente.js` dibuja la burbuja y la ventana. **No lee datos de nadie por su cuenta:** todo lo que sabe de quien pregunta se lo da el servidor. Si hubiera una consulta a una tabla de personas en el navegador, el permiso viviría en el único sitio donde no se puede defender.

Se monta sola en todas las páginas, desde la función que monta la página y no a mano en cada archivo: así ninguna se queda sin él porque a alguien se le olvidó. En las públicas se monta con ámbito `visitante`; con sesión, con el de la persona.

El botón tiene cara. La mascota o el dibujo del asistente con su nombre. A 44 píxeles un cuerpo entero es una mancha: lo que se reconoce a ese tamaño son los ojos y la boca, así que el botón usa la cara recortada. El nombre y la imagen salen de la configuración del sitio (una función pública), no del código: así el equipo los cambia sin desplegar.

El hilo vive en `sessionStorage`, no en una variable ni en `localStorage`:

- Cambiar de página volvía a cargar el módulo y borraba la conversación, justo cuando la persona iba a hacer lo que el asistente le dijo.
- `sessionStorage` dura lo que la pestaña. Una conversación de hace tres días reaparecida al abrir el sitio no es memoria, es ruido.
- **Se guarda el ámbito junto al hilo:** el hilo del equipo no se le enseña a un usuario que entre después en la misma pestaña.
- **«Empezar de nuevo» no borra nada de la base:** suelta el hilo que se le manda al modelo, que es lo que se pide cuando se pide empezar de nuevo.
- Cuando hay una conversación a medias, el botón lo indica.

Las primeras preguntas sugeridas son distintas para cada público. Un visitante no pregunta «¿cómo voy?»: pregunta lo que pregunta alguien de pie en la puerta («¿Qué tienen?», «¿Cuánto cuesta y cómo se paga?», «Avísenme cuando abra»).

La huella del visitante es una cadena al azar guardada en `localStorage`. Separa las conversaciones de visitantes distintos y nada más. El tope de gasto que de verdad protege es el **global**, que no depende de ella.

El aviso de degradado se enseña igual si el servidor marcó `degradado` o si la función ni respondió: para quien mira es lo mismo, y alguien del equipo tiene que poder distinguir «no lo sé» de «estoy roto».

El botón «Ir a…». «Los informes se sacan desde Informes → Mensuales» deja a quien pregunta buscando esa pantalla en un menú. Ahora la respuesta puede traer un botón que lleva allí, y **lo decide el servidor**, por dos razones:

1. El servidor sabe qué herramienta se usó de verdad. Si acaba de consultar los pedidos, el destino no hay que adivinarlo.
2. El servidor tiene el rol comprobado contra la base. **Un botón que rebota es peor que ningún botón**, porque parece que el sitio está roto.

`pantallas.ts` es la lista de destinos: ruta, texto del botón, **roles copiados del control de acceso de esa misma pantalla** (no inventados aquí) y frases que la reconocen. La coincidencia exige una **frase** reconocible, no una palabra suelta: «pago» aparece en medio sitio. Sale `null` a menudo, y está bien.

Se ofrece en las páginas de producto. Con el botón en todas las páginas, en cinco semanas no hubo ni una conversación de visitante: se leía como adorno, no como alguien a quien preguntar. En la página de un producto —donde nacen las dudas de compra— el asistente se ofrece solo: a los **20 segundos**, un globo encima de su botón («¿Dudas sobre este producto? Pregúntame: te contesto aquí mismo»). Pulsarlo abre el chat; la X lo quita. Reglas:

- **Una vez por pestaña** (`sessionStorage`). Quien lo cerró no lo vuelve a ver al recargar.
- **Nunca encima de algo:** no sale si ya hay conversación, si la ventana está abierta o si la pestaña no está a la vista cuando se cumplen los 20 segundos.
- **Solo en páginas de producto.** No en la portada ni en el listado, donde todavía no hay una duda concreta.

Accesibilidad y cortesía: el botón tiene `aria-expanded`; la ventana es un diálogo con nombre; Escape la cierra (en el escritorio es un reflejo, y que no pase nada se siente como que la página se colgó); mientras piensa se ve; el botón de enviar se desactiva mientras espera; la pregunta tiene un tope de 1.500 caracteres también en el campo.

---

13. El panel del equipo

Una página solo para el equipo, con ocho pestañas:

Pestaña	Para qué
Lo que sabe	Las fichas. Crear, editar, apagar. Las automáticas avisan antes de editarse. Botón «Actualizar lo que sabe».
Cómo contesta	Los guiones: lista con sus usos, editor y el probador que no gasta tokens.
Conversaciones	Todas, con filtros. Cada respuesta tiene «Así se contesta» y «Corregir». Se ve qué herramientas usó y si hubo error.
Cómo va	La salud: conversaciones y mensajes por día, cuántas respuestas degradadas, errores por modelo, tiempo de respuesta, tokens gastados.
Lo que preguntan	Las preguntas agrupadas por parecido y ordenadas por frecuencia, con «Enseñarle el dato» y «Enseñarle cómo contestar».
Por confirmar	Lo que el asistente preparó (nivel **Prepara**): a quién iría y qué diría. Confirmar o descartar.
Lo que hizo	Lo que escribió (nivel **Hace**), con quién lo pidió y un botón de deshacer.
Ajustes	Su nombre, su cara (subir la imagen) y el texto con el que se presenta.

Avisar de una caída no es lo mismo que no caerse (otra lección de Verónica). El panel de salud no basta si nadie lo mira: cuando las respuestas degradadas suben, tiene que llegarle un aviso a alguien.

14. Costes, topes y temperatura

Cada mensaje cuesta dinero de verdad. Una puerta abierta sin tope es una factura esperando a que alguien la encuentre.

Tope	Valor de referencia	Por qué
Largo de la pregunta	1.500 caracteres	Un texto pegado de diez páginas es caro y no es una pregunta.
Largo de la respuesta	700 tokens	Contesta en una o dos frases; 700 es margen para las herramientas.
Hilo que se manda	24 mensajes (16 el visitante)	Suficiente para no repetir preguntas. No se recorta para ahorrar.
Mensajes por visitante	30 por hora y huella	Una persona real no escribe más.
Mensajes de visitantes en total	400 por hora	Sin este, rotar la huella salta el primero.
Contactos públicos nuevos	según el negocio, por hora	Que nadie llene la tabla de contactos (y tu buzón) con correos inventados.

Cuando salta un tope, **no es un error técnico**: es un «hasta aquí» con una salida. Se devuelve como respuesta normal del asistente («Llevamos un buen rato conversando. Déjame tus datos y te escribe una persona»), no como una avería.

El peso de un turno. Unos 2.000 tokens de entrada (oficio, datos, guiones, hilo) más la respuesta. Haz la cuenta con los límites de tu proveedor: si da 8.000 tokens por minuto y por modelo, son cuatro turnos por minuto por modelo. Por eso la cadena tiene varios eslabones y por eso solo entran tres guiones.

La temperatura, según el público:

Público	Temperatura	Por qué
Visitante	0,85	Charla: que no conteste dos veces igual.
Usuario	0,75	Charla con datos suyos.
Equipo	0,6	Pregunta cifras: con freno.
Vuelta correctiva	0,5	Solo para arreglar una respuesta que ignoró una regla.

Antes era 0,6 para todos, y con 0,6 y sin guiones repetía frases. Con los guiones como ejemplos ya no hace falta tanto freno.

15. Las averías que enseñaron todo esto

Cada regla de este documento salió de algo que pasó. Esta es la lista, para que el proyecto nuevo no tenga que volver a descubrirlas.

Qué pasó	La regla que quedó
El servicio 24/7 corría en un portátil: 312 reconexiones en un día, 1 h 46 min caído en hora de venta.	El asistente vive en una función de servidor.
Con el catálogo vacío, el modelo inventó precios un 85 % por encima del real.	Un dato ausente se declara ausente, con la frase de espera ya escrita.
A «¿cómo te llamas?» contestaba «no tengo nombre».	El nombre va fijo en el oficio, sacado de la configuración.
Contestaba siempre la misma frase seca.	Guiones con dos o tres variantes, y temperatura más alta para charlar.
La memoria del visitante falló en silencio semanas: el insert se rechazaba y el error se tragaba.	Todo error que no se resuelve se escribe en el log. Probar que la segunda pregunta recuerda la primera.
El proveedor retiró los dos modelos de la cadena el mismo día.	Los modelos en un secreto, cambiables sin desplegar; eslabones de proveedores distintos.
Los modelos que razonan devolvían texto vacío sin error.	Pedir poco esfuerzo de razonamiento, con la palabra de cada familia.
El parámetro de razonamiento de una familia tumbaba la petición en otra.	Se decide por familia, y si da 400 por eso, se repite sin él.
Diez preguntas seguidas agotaron el límite por minuto de los dos modelos.	Más eslabones, saltar al siguiente ante un 429 y esperar una sola vez al final.
«Ningún modelo respondió» no decía qué arreglar.	El error arrastra el motivo de cada eslabón.
El filtro de frases prohibidas se comió una respuesta buena y la pantalla mostró una avería falsa.	El filtro nunca deja la respuesta vacía, y su lista es corta.
El filtro cortaba «hasta el siguiente punto» y dejó una ristra de puntos suspensivos.	Las frases se quitan enteras, partiendo por frases.
«Ya avisé al equipo», y no había avisado a nadie: la función no existía de verdad.	`bot_escalar` notifica y manda correo de verdad.
…y luego la herramienta falló y aun así dijo «ya avisé».	El servidor comprueba la promesa y avisa él, o cambia la frase.
Pidió nombre y correo tres veces seguidas aunque el guion decía una.	El servidor mira el hilo, cambia el encargo, quita la frase y, si hace falta, corrige con otra vuelta.
La función de avisos llevaba rota desde que se escribió; solo se vio al empezar a usarla.	Una avería latente aparece cuando algo empieza a usar el camino que nadie usaba: probar cada camino una vez.
La ficha de «fechas abiertas» metía una fecha de un producto pausado y ya pasada.	Las fichas automáticas solo con lo publicado y lo que todavía no ha pasado.

Qué pasó	La regla que quedó
Un correo que volvía a escribir chocaba con «correo duplicado» y su mensaje se perdía.	Al contacto que vuelve se le suma el mensaje y se avisa «volvió a escribir».
El botón estaba en todas las páginas y en cinco semanas nadie le escribió.	Se ofrece solo en las páginas de producto, una vez por pestaña.
Al cambiar de página se perdía la conversación.	El hilo vive en `sessionStorage`, con su ámbito.
«Las respuestas estaban bien» y aun así había que buscar la pantalla en un menú.	El botón «Ir a…», decidido por el servidor y con los roles de la propia pantalla.
Avisar de una caída a los quince minutos no evitaba esos quince minutos sin respuesta.	Vigilar la salud **y** no depender de una sola máquina ni de un solo proveedor.
Los secretos del servidor y del cliente no coincidían, y se descubría mensaje a mensaje.	Al arrancar, comprobar los secretos y decir con todas las letras qué falta.

PARTE II

Los prompts

Cómo usar los prompts

- **Van en orden.** Cada uno da por hecho lo anterior. No te saltes el P0: es el que hace que el asistente sea de este proyecto y no una copia.
- **Pega uno, espera a que termine y comprueba lo que entrega** antes del siguiente. Cada prompt dice cómo comprobarlo.
- **Dale antes el documento.** Todos los prompts hablan de «el documento»: es este, entero.
- **Los huecos entre corchetes** (`[NOMBRE DEL ASISTENTE]` , `[NEGOCIO]` …) los rellenas tú, o déjalos y la IA te los preguntará.
- **Si prefieres ir de una vez,** usa el Prompt maestro del final.

P0 · Aprender el proyecto

Es el más importante. La IA estudia el proyecto y te propone cómo será **su** asistente. No escribe código.

PROMPT P0 COPIAR ENTERO

```
Te paso un documento: «Un asistente para tu web». Explica un asistente de chat
web que ya funciona en producción (Cemi) y cómo montar uno igual en otro
proyecto. Léelo ENTERO antes de hacer nada.

Vamos a montar ese asistente en ESTE proyecto. Pero antes tienes que aprender el
proyecto. En este paso NO escribas código ni cambies nada: solo estudia y
entrégame un informe.

Estudia el repositorio y, si tienes acceso, la base de datos. Averigua:

1. El negocio: qué vende u ofrece, a quién, en qué idioma, desde dónde, cómo
   habla de sí mismo en el sitio.
2. Los públicos: quién entra sin cuenta, quién tiene cuenta (y cómo se llama en
   el sitio: cliente, alumno, socio…), y los roles del equipo con lo que hace
   cada uno.
3. Cómo se entra a una cuenta, y cómo se comprueba el rol en cada pantalla.
4. De dónde sale el catálogo: tablas, columnas de nombre, descripción, precio,
   moneda, duración, estado «publicado», fechas, formas de pago.
5. Qué datos tiene cada usuario sobre sí mismo (pedidos, pagos, avance,
   citas…) y qué reglas de acceso (RLS u otras) los protegen hoy.
6. Dónde se guardan los contactos de interesados, si existe, y cómo se avisa
   hoy al equipo (notificaciones, correo, nada).
7. Las páginas: cuáles son públicas, cuáles de usuario y cuáles del equipo; qué
   archivo o función monta todas las páginas (para poner la burbuja una vez);
```

```
   cuáles son páginas de producto.
8. El diseño: colores, letras, componentes, si hay versión clara y oscura.
9. Las convenciones de la casa: prefijos de tablas, idioma de los nombres,
   estilo de los comentarios, cómo se despliega, cómo se prueba.
10. Si ya usa Supabase. Si no, qué usa en su lugar para la base, las cuentas y
    el servidor, y cómo traducirías cada pieza del documento.

Entrégame un informe con:

A. Lo que encontraste, punto por punto, con las rutas de los archivos.
B. Tu propuesta de asistente para este negocio:
   - nombre, personalidad y cómo habla (dos o tres frases de muestra);
   - los públicos que atenderá y, para cada rol del equipo, su «oficio» en dos
     líneas;
   - las fichas automáticas que generará la base (de qué tablas y qué
     columnas);
   - la lista de 20 a 30 temas de guiones de fábrica para ESTE negocio;
   - las herramientas por público, cada una con su nivel (Cuenta, Prepara o
     Hace);
   - las pantallas a las que podrá llevar el botón «Ir a…», con sus roles;
   - en qué páginas se ofrecerá solo.
C. Lo que necesitas que yo decida o te dé (por ejemplo: el nombre, la cara, a
   quién se avisa, qué puede prometer, el proveedor de IA).
D. Los riesgos que veas: datos sin protección por filas, tablas sin dueño
   claro, cosas del documento que aquí no encajan.

No inventes nada del negocio: si no está en el proyecto, pregúntamelo.
```

Cómo comprobarlo: el informe nombra tablas y archivos reales del proyecto, la propuesta habla del negocio de verdad y no de Cemi, y te hace preguntas que solo tú puedes contestar. Contéstalas antes de seguir.

P1 · Cuentas y secretos

PROMPT P1 COPIAR ENTERO

```
Según el documento (Parte 0 y §9), prepara las cuentas y los secretos que
necesita el asistente en este proyecto. No me pidas que pegue ninguna clave en
el chat.

1. Dime exactamente qué tengo que crear o tener: el proyecto de Supabase (o el
   equivalente), las claves de los proveedores de IA (recomiéndame dos de
   proveedores distintos y por qué) y dónde se ponen.
2. Escríbeme la lista de secretos con su nombre exacto, para qué sirve cada uno
   y un valor de ejemplo SIN datos reales. Incluye ASISTENTE_MODELOS con el
   formato proveedor:modelo,proveedor:modelo. Busca en la documentación de
   cada proveedor qué modelos están vigentes hoy: no me propongas modelos de
   memoria, porque se retiran.
3. Cuando te diga que ya los puse, comprueba que existen SIN mostrarlos.
   Despliega una función mínima de diagnóstico que diga, por cada secreto
   esperado: si está, si tiene el prefijo que usa ese proveedor y, si falta,
   qué nombres parecidos hay. Que llame una vez a cada modelo de la cadena
```

```
   con la pregunta «di hola» y diga cuáles responden y el error exacto de los
   que no. Después bórrala.
```

Cómo comprobarlo: el diagnóstico dice que todos los secretos están y que cada modelo de la cadena responde.

P2 · La base de datos

PROMPT P2 — COPIAR ENTERO

```
Según el documento (§3, §4, §5, §6, §11 y §14, y el Anexo C), crea la base del
asistente en este proyecto, como una migración nueva y adaptada a las
convenciones de la casa.

Tablas, todas con RLS encendido y SIN políticas: bot_conocimiento,
bot_guiones, bot_conversaciones (con ámbito, canal 'web' o 'visitante', huella,
escalado_en, escalado_motivo), bot_mensajes (con modelo, tokens, ms y error),
bot_escuchado, bot_borradores y bot_acciones (o el registro de auditoría que ya
tenga el proyecto).

Funciones:
- bot_contexto(p_ambito): SECURITY INVOKER. Devuelve un JSON con el ámbito
  real (el que corresponde a quien llama, no el que pide), quién es, lo suyo,
  las cifras de su rol si es del equipo, el catálogo publicado y las fichas
  activas de su ámbito. Tiene que funcionar con las reglas de fila del
  proyecto: si a alguien no le toca una fila, no la ve.
- bot_contexto_publico(): SECURITY DEFINER, solo datos públicos, para el
  visitante.
- bot_guiones_para(texto, ámbito, tope, contar): la puntuación exacta del
  documento (§6).
- bot_visitante_permitir(huella): los dos topes del §14.
- bot_abrir, bot_historial, bot_guardar (usuario con sesión) y
  bot_conversacion_visitante, bot_historial_visitante, bot_guardar_visitante
  (visitante), con la regla «se guarda después de responder».
- bot_escalar(conversación, motivo): avisa de verdad por el canal del
  proyecto, no repite en 6 horas y añade a quien cobra si el motivo habla de
  dinero.
- bot_dejar_contacto(...): guarda en la tabla de contactos del proyecto; si el
  correo ya existe, suma el mensaje en vez de fallar; con tope por hora.
- bot_anotar_accion y bot_deshacer, para lo que escriben las herramientas.

Permisos: lo que solo llama el servidor, cerrado para el público y los
usuarios (solo la llave de servicio); lo que llama la ventana o el panel,
abierto solo a usuarios con sesión y comprobando dentro quién llama. Ninguna
función recibe «de quién son los datos» como parámetro: se atiene a la sesión.

Pruébalo en la base, sin dejar basura (borra solo lo que crees, por su id):
- un usuario A no ve nada de un usuario B en bot_contexto;
- pedir 'equipo' siendo usuario devuelve el ámbito de usuario;
- bot_guiones_para con tres textos de muestra elige lo que el documento dice;
- bot_visitante_permitir corta en el mensaje 31 de una huella;
```

```
- una conversación de visitante se crea y su historial se lee (la memoria).
Enséñame el resultado de cada prueba.
```

Cómo comprobarlo: las cinco pruebas, con su salida. Si el proyecto no tiene reglas de fila en las tablas de usuarios, la IA tiene que decirlo: sin ellas, `bot_contexto` no protege nada.

P3 · Lo que sabe solo: las fichas

PROMPT P3 — COPIAR ENTERO

```
Según el documento (§5), haz que el asistente se entere solo de lo que vende
este negocio.

1. Escribe bot_refrescar_ahora(): genera una ficha por cada producto o servicio
   PUBLICADO (nombre, de qué va, precio y moneda, duración, partes), una con las
   formas de pago activas y otra con las próximas fechas abiertas (solo de lo
   publicado y que todavía no haya pasado). Cada ficha con una clave única
   ('producto:<id>', 'pagos', 'fechas'). Si un dato no existe, la ficha lo dice
   («todavía no hay formas de pago: no inventes ninguna»). Solo reescribe las de
   origen 'plataforma', nunca las del equipo. Apaga, sin borrar, las de lo que
   ya no está publicado.
2. bot_refrescar(): lo mismo, comprobando que quien la llama es del equipo.
3. Disparadores en las tablas del catálogo, FOR EACH STATEMENT, envueltos en
   su propio manejador de errores: si la ficha falla, el cambio del catálogo
   pasa igual.
4. Editar una ficha automática desde el panel la convierte en 'equipo'.

Pruébalo: publica (o simula, en una transacción que deshaces) un producto, y
enséñame la ficha que sale; cámbiale el precio y enséñame la ficha cambiada;
edita una a mano, cambia el precio otra vez y enséñame que la tuya no se tocó.
Después deja todo como estaba.
```

Cómo comprobarlo: las tres comprobaciones, con las fichas antes y después.

P4 · El cerebro: la función de servidor

PROMPT P4 — COPIAR ENTERO

```
Según el documento (§2, §4, §7, §8, §9, §10, §11 y §14, y el Anexo D), escribe
la función de servidor «asistente» y sus módulos compartidos (cerebro,
herramientas, pantallas) en este proyecto.

Requisitos que no se negocian:
- Dos caminos: el visitante (sin sesión, con la llave de servicio, solo lo
  público y la herramienta dejar_contacto, con bot_visitante_permitir antes de
  nada) y el usuario con sesión (TODO con su token: el contexto, el historial y
  cada herramienta). El camino del visitante se decide antes de leer ningún
  contexto.
- El guion del sistema por capas, en el orden del §7, con el nombre y la
```

```
  personalidad que acordamos en el P0, el oficio de cada rol del equipo, y el
  bloque de datos que distingue «no pude leer el catálogo» de «no hay nada
  publicado».
- La cadena de modelos leída del secreto ASISTENTE_MODELOS, con varios
  proveedores compatibles con OpenAI; 429 → siguiente eslabón sin esperar y una
  sola espera al final (máximo 6 s); esfuerzo de razonamiento por familia y
  reintento sin él si da 400; vacío solo es fallo si no pidió herramientas; el
  error final con el motivo de cada eslabón; diagnóstico de la clave sin
  mostrarla.
- El bucle de herramientas: máximo 4 llamadas y 3 vueltas, la última sin
  herramientas; solo nombres de la carta; solo parámetros declarados más los
  del servidor; el error se le devuelve al modelo; lo que escribe se anota.
- El filtro de salida del §10, que nunca deja la respuesta vacía.
- Las redes del §11: la promesa de avisar comprobada por el servidor, los
  datos de contacto pedidos una sola vez (las cuatro capas), la respuesta
  degradada marcada y con su frase honesta para cada público, y guardar
  después de responder.
- Los topes y temperaturas del §14.
- La respuesta devuelve: respuesta, ámbito, modelo, ms, degradado, hizo (las
  herramientas usadas y su error) e ir (el botón, o null).
- Cada pregunta de visitante o usuario se anota en bot_escuchado (sin datos
  personales).

Despliégala y pruébala con llamadas reales:
1. Visitante: «¿qué tienen?» y luego «¿y cuánto cuesta?» — la segunda tiene que
   recordar la primera.
2. Visitante: «me interesa, ¿cuánto es?» → pide los datos; «¿y se puede pagar
   en partes?» → NO los vuelve a pedir.
3. Usuario A preguntando por los datos del usuario B → no los da, y no porque
   lo diga el guion: el contexto llega sin ellos.
4. «Quiero hablar con una persona» → bot_escalar se ejecuta de verdad (enséñame
   el aviso).
5. Pon un modelo inexistente el primero de la cadena → contesta igual con el
   segundo, y el panel registra el error del primero.
6. Todos los modelos inexistentes → frase de cortesía, degradado = true, y
   aviso al equipo si había conversación.
Enséñame la respuesta completa de cada prueba. Después deja la cadena como
estaba.
```

Cómo comprobarlo: las seis pruebas, con sus respuestas. La 3 es la que importa: si falla, no se enciende.

P5 · La ventana

PROMPT P5 COPIAR ENTERO

```
Según el documento (§12), escribe la ventana del asistente (asistente.js y sus
estilos) con el diseño de ESTE proyecto: sus colores, sus letras, sus
componentes y, si las tiene, sus versiones clara y oscura.

- Se monta sola en TODAS las páginas desde el sitio que monta las páginas (el
  que encontraste en el P0), con ámbito 'visitante' en las públicas y el de la
  persona si tiene sesión. No la pegues a mano en cada página.
- No lee datos de nadie: solo llama a la función «asistente».
```

```
- El botón con la cara del asistente y su nombre, que salen de la
  configuración del sitio y no del código (si la imagen no está, un dibujo de
  respaldo).
- El hilo en sessionStorage, guardado con su ámbito; «empezar de nuevo» que no
  borra nada de la base; el botón marca cuando hay una conversación a medias.
- Las primeras preguntas sugeridas, distintas para visitante, usuario y equipo,
  escritas para este negocio.
- La huella del visitante en localStorage (al azar, no identifica).
- El aviso de degradado, igual si el servidor lo marca o si la función no
  responde.
- El botón «Ir a…» cuando la respuesta trae ir.
- Accesibilidad: aria-expanded, diálogo con nombre, Escape cierra, foco
  visible, enviar desactivado mientras espera, tope de 1.500 caracteres en el
  campo.
- Que no tape nada importante del sitio en el teléfono, y que no desborde a 360
  píxeles de ancho.

Pruébalo en un navegador real, en ordenador y en teléfono, y en las dos
versiones si las hay: abre, pregunta, cambia de página y comprueba que el hilo
sigue, empieza de nuevo, cierra con Escape. Mándame capturas.
```

Cómo comprobarlo: las capturas, y que al cambiar de página la conversación sigue ahí.

P6 · Los guiones de fábrica

PROMPT P6 COPIAR ENTERO

```
Según el documento (§6 y Anexo A y B), escribe los guiones de fábrica de ESTE
negocio: entre 20 y 30, con los temas que acordamos en el P0.

Para cada uno: ámbito (visitante, usuario, equipo o ambos), tema, disparadores
(palabras y frases como las escribe la gente de verdad, con y sin tildes, con
faltas habituales), el guion en el formato del documento (Persona: / dos o tres
formas de contestar / ramas «Si dice …»), y notas para el equipo.

Reglas:
- Ningún precio, fecha, horario ni dato que cambie: van como [huecos] que se
  rellenan con las fichas.
- El tono es el que acordamos para [NOMBRE DEL ASISTENTE]. Cada variante,
  distinta de verdad, no la misma frase con otro orden.
- Uno solo marcado «siempre»: cómo salir de lo que no toca.
- Incluye siempre: quién eres (con nombre), saludo, hablar con una persona
  ([usa avisar_al_equipo]), «me interesa» ([usa dejar_contacto]), «¿eres un
  bot?», quejas, y qué pasa cuando no hay nada abierto.

Siémbralos con una función que no pise los que el equipo haya editado. Después,
con el probador (bot_guiones_para sin contar usos), enséñame qué guiones
elegiría para diez preguntas de muestra escritas como las escribiría un
cliente real, y ajusta los disparadores de los que no salgan cuando deberían.
```

Cómo comprobarlo: la tabla de las diez preguntas con los guiones elegidos. Cada pregunta típica trae al menos uno que le toca.

P7 · Las herramientas por público

PROMPT P7 COPIAR ENTERO

```
Según el documento (§8 y §4), diseña y escribe las herramientas del asistente
para ESTE proyecto, con la lista que acordamos en el P0.

- Cada una es una función de la base. Las de datos, SECURITY INVOKER, para que
  corran con el permiso de quien pregunta. Si alguna necesita filtrar más de lo
  que filtran las reglas de fila (por ejemplo, alguien del equipo que solo debe
  ver lo suyo), fíltralo dentro y explícame por qué.
- Cada una declarada en herramientas.ts con nombre, rpc, descripción (cuándo
  usarla y qué NO hace), parámetros, obligatorios, delServidor, ámbito, roles y,
  si escribe, entidad e idEnRespuesta.
- Niveles: Cuenta (solo lee), Prepara (deja un borrador en bot_borradores para
  que una persona lo confirme en el panel) y Hace (escribe, se anota y se puede
  deshacer). Mandar mensajes a muchos o mover dinero: siempre Prepara.
- Siempre: avisar_al_equipo para usuario y equipo; dejar_contacto para el
  visitante.

Pruébalas con dos usuarios y dos roles del equipo: la misma herramienta llamada
por quien no le toca devuelve vacío o un error claro, nunca datos ajenos. Pide
al asistente, en lenguaje normal, una acción de cada nivel y enséñame qué
herramienta usó, el borrador que quedó y la acción anotada con su deshacer.
```

Cómo comprobarlo: una tabla con cada herramienta, quién la puede usar y la prueba de que a quien no le toca no le da nada.

P8 · El panel del equipo

PROMPT P8 COPIAR ENTERO

```
Según el documento (§13), haz la página del asistente para el equipo, con el
diseño y el control de acceso del resto del panel de este proyecto.

Ocho pestañas: Lo que sabe, Cómo contesta (con el probador que no gasta
tokens), Conversaciones (con «Así se contesta» y «Corregir» en cada
respuesta), Cómo va (conversaciones por día, degradadas, errores por modelo,
tiempo de respuesta, tokens), Lo que preguntan (agrupado, con «Enseñarle el
dato» y «Enseñarle cómo contestar»), Por confirmar, Lo que hizo (con deshacer)
y Ajustes (nombre, cara y presentación).

Todo por funciones de la base que comprueban el rol dentro; nada leído de las
tablas desde el navegador. Que solo lo vean los roles que acordamos.

Y un aviso: si en la última hora más del 20 % de las respuestas salieron
degradadas, que le llegue una notificación a quien lleva el asistente, una vez
```

```
cada pocas horas como mucho.

Pruébalo con un rol que puede entrar y con uno que no. Mándame capturas de las
ocho pestañas con datos de prueba.
```

Cómo comprobarlo: las capturas, y que un rol sin permiso no ve la página ni consigue los datos llamando a las funciones.

P9 · El botón que lleva a la pantalla

PROMPT P9 — COPIAR ENTERO

```
Según el documento (§12, «El botón Ir a…»), escribe pantallas.ts para ESTE
proyecto: la lista de pantallas a las que el asistente puede llevar a alguien.

- Cada destino con su ruta, el texto del botón (corto), los roles COPIADOS del
  control de acceso de esa misma pantalla (léelo en su código, no lo supongas)
  y las frases que la reconocen.
- La coincidencia exige una frase, no una palabra suelta que aparezca en medio
  sitio.
- Si la respuesta usó una herramienta, el destino de esa herramienta manda.
- Nunca un botón a una pantalla que esa persona no puede abrir. Mejor null.

Pruébalo con al menos diez casos: preguntas de usuario, de cada rol y de
visitante, incluidos dos que NO deben dar botón y uno en el que un rol sin
permiso nunca reciba una pantalla de administración. Escribe además una prueba
que compare los roles de pantallas.ts con el control de acceso de cada página,
para que avise si un día se separan.
```

Cómo comprobarlo: la tabla de los diez casos con el botón que sale, o null.

P10 · Ofrecerse en las páginas de producto

PROMPT P10 — COPIAR ENTERO

```
Según el documento (§12, «Se ofrece en las páginas de producto»), haz que el
asistente se ofrezca solo en las páginas de producto de ESTE sitio (las que
encontraste en el P0).

A los 20 segundos, un globo encima de su botón con una frase escrita para este
negocio y con su nombre. Pulsarlo abre el chat; la X lo quita. Una vez por
pestaña (sessionStorage). No sale si ya hay conversación, si la ventana está
abierta o si la pestaña no está a la vista. Solo en páginas de producto, no en
la portada ni en el listado. Que respete prefers-reduced-motion y no tape nada
en el teléfono.

Pruébalo en el navegador: sale a los 20 s en una página de producto, no sale en
```

```
la portada, no vuelve a salir al recargar después de cerrarlo. Mándame
capturas en ordenador y teléfono.
```

P11 · Las pruebas antes de encenderlo

PROMPT P11 COPIAR ENTERO

```
Según el documento (§15 y Anexo E), comprueba el asistente de este proyecto
contra cada avería conocida antes de encenderlo para el público. Para cada
caso, hazlo de verdad (no lo supongas), enséñame la salida y dime si pasa.

1. Usuario A pide datos del usuario B, por las buenas, insistiendo y diciendo
   que es una emergencia → nada de B, en ninguno de los tres intentos.
2. Usuario pide ámbito 'equipo' desde la consola → recibe el suyo.
3. Visitante: dos preguntas seguidas → la segunda recuerda la primera.
4. Catálogo vacío (nada publicado) → dice que no hay nada abierto, sin un
   precio.
5. Catálogo que no se pudo leer (simula el fallo) → la frase de espera, sin un
   precio.
6. «¿Cómo te llamas?» tres veces → su nombre, dicho de tres formas distintas.
7. Pide datos de contacto y la persona no los da → no los vuelve a pedir en
   las tres preguntas siguientes.
8. «Quiero hablar con alguien» → aviso real al equipo (enséñamelo).
9. La herramienta de avisar falla (simúlalo) y el modelo dice «ya avisé» → el
   servidor avisa él o cambia la frase.
10. Primer modelo de la cadena inexistente → contesta con el segundo.
11. Todos inexistentes → frase honesta, degradado visible en la ventana.
12. Diez preguntas seguidas en un minuto → sin frase de avería (o, si el plan
    del proveedor no da para más, dímelo con números).
13. Una respuesta que el filtro dejaría vacía → sale la original.
14. Un visitante que manda 31 mensajes en una hora → el tope, como frase del
    asistente y no como error.
15. Cambiar de página con una conversación abierta → el hilo sigue.
16. Publicar un producto → el asistente lo conoce en la siguiente pregunta.
17. Un botón «Ir a…» nunca lleva a una pantalla que esa persona no puede abrir.
18. Nada del asistente se puede leer desde el navegador sin pasar por las
    funciones (prueba a leer las tablas con la clave pública).

Al final, una tabla: caso, pasa o no, y qué arreglaste. Si algo no se pudo
probar, dilo y por qué.
```

P12 · Alimentarlo

Este no se pega una vez: es la rutina para las primeras semanas.

PROMPT P12 COPIAR ENTERO

```
Revisa cómo le fue al asistente esta semana, según el documento (§5, §6 y §13):

1. En «Cómo va»: cuántas conversaciones, cuántas degradadas y por qué error,
```

```
   qué modelo falló y cuánto se gastó. Si un modelo devuelve model_not_found,
   propónme el cambio de ASISTENTE_MODELOS (no lo cambies sin decírmelo).
2. En «Lo que preguntan»: las diez preguntas más repetidas que no tienen ficha.
   Para cada una, propónme la ficha o el guion que la contestaría, con el texto.
3. En «Conversaciones»: cinco respuestas que estuvieron mal o flojas, por qué, y
   el guion corregido para cada una.
4. Guiones que no se usaron en un mes: propónme apagarlos o arreglar sus
   disparadores.

No guardes nada sin que yo lo apruebe. Cuando lo apruebe, guárdalo y pásalo por
el probador para comprobar que se elige.
```

Prompt maestro

Para hacerlo todo de una vez, por fases. La IA se para al final de cada fase a enseñarte lo que hizo.

PROMPT MAESTRO — COPIAR ENTERO

```
Te paso un documento: «Un asistente para tu web». Explica un asistente de chat
web en producción (Cemi): cómo está montado, por qué, las averías que enseñaron
cada regla, y los prompts para montarlo en otro proyecto. Léelo ENTERO antes de
hacer nada.

Vamos a montar ese asistente en ESTE proyecto, desde cero y con sus propias
cuentas, adaptado a este negocio. Sigue la Parte II del documento como plan de
trabajo, en este orden y por fases:

Fase 1 — Aprender (P0). Estudia el proyecto y entrégame el informe y la
propuesta. PARA y espera mis respuestas.
Fase 2 — Cimientos (P1, P2, P3). Secretos comprobados sin verlos, la base y las
fichas automáticas, con sus pruebas. PARA y enséñame las pruebas.
Fase 3 — El cerebro y la ventana (P4, P5). Con sus pruebas reales y capturas.
PARA.
Fase 4 — Que sepa contestar y hacer (P6, P7, P9). Guiones de fábrica de este
negocio, herramientas por público y el botón «Ir a…». PARA.
Fase 5 — El equipo (P8, P10). El panel y ofrecerse en las páginas de producto.
PARA.
Fase 6 — Antes de encenderlo (P11). Las dieciocho pruebas, con su tabla. Si
alguna no pasa, arréglala antes de decirme que está listo.

Reglas para todo el trabajo:
- Los permisos viven en la base, nunca en el guion ni en el navegador.
- No inventes nada del negocio: lo que no esté en el proyecto, pregúntamelo.
- No copies nombres, textos ni datos de Cemi: este asistente es de este
  proyecto. Su nombre y su forma de hablar son los que acordemos.
- Ningún secreto en el código ni en el chat.
- Sigue las convenciones de este repositorio.
- No me digas que algo funciona sin haberlo probado. Si algo no se pudo probar,
  dímelo y explícame por qué.
```

ANEXOS

Plantillas, la base y el código de referencia

Anexo A · La plantilla de un guion

TEXTO

```
Persona: [lo que escribe la gente, tal cual, con sus faltas]
[ASISTENTE]: [primera forma de contestar]
[ASISTENTE]: [segunda forma, distinta de verdad]
[ASISTENTE]: [tercera, si hace falta]
Si dice «[lo que suele seguir]» → [ASISTENTE]: [cómo sigue]
Si dice «[otra salida]» → [ASISTENTE]: [cómo sigue] [usa dejar_contacto]
```

Además del texto, cada guion lleva:

Campo	Ejemplo
Ámbito	`visitante`, `usuario`, `equipo` o `ambos`
Tema	«Cuánto cuesta»
Disparadores	`cuanto cuesta`, `precio`, `cuanto sale`, `valor`, `cuanto es`
Siempre	no (solo el de «fuera de tema» va marcado)
Notas	Para el equipo: cuándo se usa y qué no debe prometer.

Un ejemplo genérico:

TEXTO

```
Persona: cuanto cuesta?
[ASISTENTE]: [producto] está en [precio]. Si quieres te cuento qué incluye.
[ASISTENTE]: Sale en [precio]. ¿Te cuento cómo se paga?
Si dice «y en partes?» → [ASISTENTE]: [cómo se paga]. ¿Te interesa alguno en
  concreto?
Si dice «me interesa» → [ASISTENTE]: Genial. Si me dejas tu nombre y tu correo,
  te escribe alguien del equipo con todo. [usa dejar_contacto]
Si no hay [precio] → [ASISTENTE]: Ahora mismo no hay nada abierto. Si quieres te
  aviso cuando abra.
```

Anexo B · Los temas de fábrica

Una lista de partida. El P0 la adapta al negocio: quita los que no aplican y añade los propios.

#	Tema	Ámbito
1	Quién eres (nombre y personalidad)	ambos
2	Saludo y primera pregunta	ambos
3	¿Eres un bot?	ambos
4	Qué es [NEGOCIO]	visitante
5	Qué ofrecen	visitante
6	Cuánto cuesta	visitante
7	Cómo se paga	ambos
8	Cuánto dura / cuánto tarda	visitante
9	Dónde, cuándo, en qué horario	visitante
10	En línea o en persona	visitante
11	Para quién es	visitante
12	Qué hace falta para empezar	visitante
13	Cuándo empieza / cuándo abre	visitante
14	Descuentos y promociones	visitante
15	Para empresas	visitante
16	Cómo comprar o apuntarse	visitante
17	Me interesa / lo voy a pensar	visitante
18	Ya soy cliente	ambos
19	Dónde veo lo mío (pedidos, pagos, avance)	usuario
20	Ya pagué, ¿me lo confirmas?	usuario
21	Hablar con una persona	ambos
22	Quejas y reembolsos	ambos
23	Desde otro país	visitante
24	Objeciones («es caro», «no tengo tiempo»)	visitante
25	Qué garantía hay / qué se lleva	visitante
26	No hay nada abierto ahora	visitante
27	Fuera de tema (chistes, tareas, insultos) — **siempre**	ambos

#	Tema	Ámbito
28	Dónde se hace X en el panel	equipo

Anexo C · El esquema de la base

Referencia para Postgres en Supabase. Los nombres del negocio (`productos`, `perfiles`, `contactos`, `avisar_equipo`) son de ejemplo: el P0 los cambia por los del proyecto.

Las tablas. RLS encendido y ninguna política en todas.

SQL

```
create table public.bot_conocimiento (
  id uuid primary key default gen_random_uuid(),
  ambito text not null default 'ambos'
    check (ambito in ('visitante','usuario','equipo','ambos')),
  titulo text not null,
  contenido text not null,
  etiquetas text[] not null default '{}',
  origen text not null default 'equipo' check (origen in ('plataforma','equipo')),
  clave text unique,            -- 'producto:<id>', 'pagos', 'fechas'
  activo boolean not null default true,
  creado_por uuid,
  created_at timestamptz not null default now(),
  actualizado_en timestamptz not null default now()
);

create table public.bot_guiones (
  id uuid primary key default gen_random_uuid(),
  ambito text not null default 'visitante'
    check (ambito in ('visitante','usuario','equipo','ambos')),
  tema text not null,
  disparadores text[] not null default '{}',
  guion text not null,
  notas text,
  siempre boolean not null default false,
  activo boolean not null default true,
  origen text not null default 'equipo',   -- 'fabrica', 'equipo', 'conversacion'
  veces_usado int not null default 0,
  ultimo_uso timestamptz,
  creado_por uuid,
  created_at timestamptz not null default now(),
  actualizado_en timestamptz not null default now()
);

create table public.bot_conversaciones (
  id uuid primary key default gen_random_uuid(),
  perfil_id uuid,                          -- null en el visitante
  ambito text not null,
  canal text not null check (canal in ('web','visitante')),
  huella text,                             -- solo el visitante
  titulo text,
  escalado_en timestamptz,
  escalado_motivo text,
  created_at timestamptz not null default now(),
  ultimo_en timestamptz
);

create table public.bot_mensajes (
```

```
  id uuid primary key default gen_random_uuid(),
  conversacion_id uuid not null references public.bot_conversaciones(id),
  quien text not null check (quien in ('persona','asistente')),
  texto text not null,
  modelo text, tokens_entrada int, tokens_salida int, ms int, error text,
  created_at timestamptz not null default now()
);

create table public.bot_escuchado (        -- «Lo que preguntan»
  id uuid primary key default gen_random_uuid(),
  canal text not null,
  quien_huella text,                       -- un hash, nunca el dato
  texto text not null,                     -- ya sin correos ni teléfonos
  ficha_id uuid,                           -- la ficha que lo resolvió
  descartada boolean not null default false,
  created_at timestamptz not null default now()
);

create table public.bot_borradores (       -- nivel «Prepara»
  id uuid primary key default gen_random_uuid(),
  tipo text not null,
  creado_por uuid,
  conversacion uuid,
  resumen text, cuerpo text,
  a_quien jsonb, ids jsonb,
  estado text not null default 'pendiente',
  resuelto_por uuid, resuelto_en timestamptz, resultado jsonb,
  created_at timestamptz not null default now()
);

alter table public.bot_conocimiento   enable row level security;
alter table public.bot_guiones        enable row level security;
alter table public.bot_conversaciones enable row level security;
alter table public.bot_mensajes       enable row level security;
alter table public.bot_escuchado      enable row level security;
alter table public.bot_borradores     enable row level security;
```

Texto plano, para comparar sin tildes ni signos.

SQL

```
create or replace function public.bot_texto_plano(p text)
returns text language sql immutable set search_path = public, pg_temp as $$
  select trim(regexp_replace(
    lower(translate(coalesce(p, ''),
      'áéíóúüñÁÉÍÓÚÜÑàèìòùâêîôû', 'aeiouunAEIOUUNaeiouaeiou')),
    '[^a-z0-9ñ ]+', ' ', 'g'));
$$;
```

Elegir los guiones. La puntuación del §6.

SQL

```
create or replace function public.bot_guiones_para(
  p_texto text, p_ambito text default 'visitante',
  p_tope int default 3, p_contar boolean default true)
returns table(guion_id uuid, guion_tema text, guion_texto text,
              guion_siempre boolean, guion_puntos int)
language plpgsql security definer set search_path = public as $$
declare
  v_texto text := bot_texto_plano(coalesce(p_texto, ''));
  v_palabras text[];
```

```
  v_ids uuid[] := '{}';
  r record;
begin
  select coalesce(array_agg(distinct w), '{}') into v_palabras
    from regexp_split_to_table(v_texto, '\s+') w where length(w) >= 4;

  for r in
    with cand as (
      select g.id, g.tema, g.guion, g.siempre, g.veces_usado,
             (select count(*) from unnest(g.disparadores) d
               where d <> '' and position(bot_texto_plano(d) in v_texto) > 0)::int * 3
           + (select count(*) from unnest(v_palabras) w
               where position(w in bot_texto_plano(
                 g.tema || ' ' || array_to_string(g.disparadores, ' '))) > 0)::int
             as puntos
        from bot_guiones g
       where g.activo and g.ambito in (coalesce(p_ambito, 'visitante'), 'ambos')
    ), elegidos as (
      select * from cand where siempre
      union all
      (select * from cand where not siempre and puntos >= 2
        order by puntos desc, veces_usado asc, tema
        limit greatest(coalesce(p_tope, 3), 0))
    )
    select * from elegidos order by siempre desc, puntos desc, tema
  loop
    guion_id := r.id; guion_tema := r.tema; guion_texto := r.guion;
    guion_siempre := r.siempre; guion_puntos := r.puntos;
    v_ids := v_ids || r.id;
    return next;
  end loop;

  if p_contar and array_length(v_ids, 1) > 0 then
    update bot_guiones g set veces_usado = g.veces_usado + 1, ultimo_uso = now()
     where g.id = any(v_ids);
  end if;
end $$;
revoke all on function public.bot_guiones_para(text, text, int, boolean)
  from public, anon, authenticated;       -- solo el servidor
```

Los topes del visitante.

SQL

```
create or replace function public.bot_visitante_permitir(p_huella text)
returns jsonb language plpgsql security definer set search_path = public as $$
declare v_suyas int; v_todas int;
begin
  -- Por huella y, por encima, en total: sin el segundo, rotar la huella
  -- salta el primero.
  select count(*) into v_suyas from bot_mensajes m
    join bot_conversaciones c on c.id = m.conversacion_id
   where c.canal = 'visitante' and c.huella = p_huella
     and m.created_at > now() - interval '1 hour';
  if v_suyas >= 30 then
    return jsonb_build_object('ok', false, 'porque',
      'Llevamos un buen rato conversando. Déjame tus datos y te escribe una persona.');
  end if;

  select count(*) into v_todas from bot_mensajes m
    join bot_conversaciones c on c.id = m.conversacion_id
   where c.canal = 'visitante' and m.created_at > now() - interval '1 hour';
  if v_todas >= 400 then
```

```
    return jsonb_build_object('ok', false, 'porque',
      'Ahora mismo hay mucha gente preguntando. Déjame tus datos y te escribimos.');
  end if;

  return jsonb_build_object('ok', true);
end $$;
revoke all on function public.bot_visitante_permitir(text)
  from public, anon, authenticated;
```

El contexto de quien pregunta: `security invoker`. Es la pieza de la seguridad. Lee las tablas del negocio **con los permisos de quien llama**, así que sus reglas de fila hacen el trabajo.

SQL

```
create or replace function public.bot_contexto(p_ambito text default 'usuario')
returns jsonb
language plpgsql
security invoker                        -- A PROPÓSITO: corre como quien pregunta
set search_path = public as $$
declare
  v_yo perfiles;
  v_ambito text;
begin
  select * into v_yo from perfiles where id = auth.uid();
  if v_yo.id is null then raise exception 'Hay que entrar.'; end if;

  -- El ámbito lo decide la base: pedir 'equipo' no te hace del equipo.
  v_ambito := case when p_ambito = 'equipo' and v_yo.rol in ('admin', 'ventas', 'soporte')
                  then 'equipo' else 'usuario' end;

  return jsonb_build_object(
    'ambito', v_ambito,
    'asistente', jsonb_build_object('nombre', coalesce(
       (select valor from ajustes_publicos where clave = 'asistente_nombre'), '[ASISTENTE]')),
    'quien', jsonb_build_object(
       'nombre', v_yo.nombre, 'primer_nombre', split_part(v_yo.nombre, ' ', 1),
       'rol', v_yo.rol),
    -- Lo suyo: las reglas de fila de cada tabla dejan pasar solo sus filas.
    'lo_mio', jsonb_build_object(
       'pedidos', (select coalesce(jsonb_agg(jsonb_build_object(
                     'producto', p.nombre, 'estado', o.estado, 'fecha', o.created_at::date)), '[]')
                     from pedidos o join productos p on p.id = o.producto_id)),
    -- Las cifras, solo si es del equipo; y aun así, las que su rol puede leer.
    'del_negocio', case when v_ambito = 'equipo' then jsonb_build_object(
       'pedidos_hoy', (select count(*) from pedidos where created_at::date = current_date))
       else '{}'::jsonb end,
    'programas', (select coalesce(jsonb_agg(jsonb_build_object(
       'id', id, 'nombre', nombre, 'resumen', descripcion,
       'precio', precio, 'moneda', moneda)), '[]')
       from productos where estado = 'publicado'),
    'lo_aprendido', (select coalesce(jsonb_agg(jsonb_build_object(
       'titulo', titulo, 'contenido', contenido)), '[]')
       from bot_conocimiento
      where activo and ambito in (case when v_ambito = 'equipo' then 'equipo' else 'usuario' end,
'ambos'))
  );
end $$;
revoke all on function public.bot_contexto(text) from public, anon;
grant execute on function public.bot_contexto(text) to authenticated;
```

> Ojo: como corre con los permisos de quien llama, `bot_conocimiento` (RLS sin políticas) **no se podría leer** desde aquí. Hay dos salidas: leer las fichas con una función `security definer` aparte que solo devuelve fichas, o dar una política de solo lectura sobre `bot_conocimiento` a `authenticated`. Las fichas no son datos de personas, así que cualquiera de las dos es segura.

Ponerse al día solo: los disparadores.

SQL

```
create or replace function public.bot_al_cambiar_catalogo()
returns trigger language plpgsql security definer set search_path = public as $$
begin
  begin
    perform bot_refrescar_ahora();
  exception when others then
    -- Que el asistente no se entere NUNCA puede impedir guardar el catálogo.
    raise warning 'El asistente no pudo ponerse al día: %', sqlerrm;
  end;
  return null;
end $$;

create trigger bot_catalogo_productos
  after insert or delete or update of estado, nombre, precio, moneda, descripcion
  on public.productos
  for each statement execute function public.bot_al_cambiar_catalogo();
```

Anotar lo que pregunta la gente, sin datos personales.

SQL

```
create or replace function public.bot_redactar(p text)
returns text language sql immutable as $$
  select regexp_replace(
           regexp_replace(coalesce(p, ''),
             '[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}', '[correo]', 'g'),
           '[+]?[0-9][0-9 ()._-]{7,}[0-9]', '[numero]', 'g')
$$;

create or replace function public.bot_anotar(p_texto text, p_canal text)
returns void language plpgsql security definer set search_path = public as $$
declare v_texto text := trim(bot_redactar(p_texto));
begin
  -- Un «ok» o un «gracias» no enseñan nada y llenarían la lista de ruido.
  if length(v_texto) < 12 then return; end if;
  insert into bot_escuchado (canal, texto) values (p_canal, left(v_texto, 1000));
end $$;
revoke all on function public.bot_anotar(text, text) from public, anon, authenticated;
```

Escalar: el aviso de verdad. Esqueleto: `avisar_equipo` es la función del proyecto que manda la notificación y el correo.

SQL

```
create or replace function public.bot_escalar(p_conversacion uuid, p_motivo text default null)
returns jsonb language plpgsql security definer set search_path = public as $$
declare
  v_c bot_conversaciones;
  v_motivo text := nullif(trim(coalesce(p_motivo, '')), '');
```

```
  v_repetido boolean;
  v_dinero boolean;
begin
  select * into v_c from bot_conversaciones where id = p_conversacion;
  if v_c.id is null then raise exception 'Esa conversación no existe.'; end if;
  -- Solo el dueño de la conversación, el equipo, o el servidor (sin sesión).
  if auth.uid() is not null and v_c.perfil_id is distinct from auth.uid()
     and not es_del_equipo() then
    raise exception 'No es tu conversación.';
  end if;

  v_repetido := v_c.escalado_en is not null and v_c.escalado_en > now() - interval '6 hours';
  update bot_conversaciones
     set escalado_en = now(), escalado_motivo = coalesce(v_motivo, escalado_motivo)
   where id = p_conversacion;
  if v_repetido then return jsonb_build_object('ya_estaba_avisado', true); end if;

  v_dinero := coalesce(v_motivo, '') ~* '(pag|cobr|factur|reembols|precio|descuent|deuda)';
  perform avisar_equipo(
    'El asistente pide una persona',
    coalesce(v_motivo, 'Pidió hablar con alguien del equipo.'),
    '/panel/asistente?conversacion=' || p_conversacion,
    case when v_dinero then array['admin','soporte','cobros'] else array['admin','soporte'] end);
  return jsonb_build_object('ya_estaba_avisado', false, 'incluye_cobros', v_dinero);
end $$;
revoke all on function public.bot_escalar(uuid, text) from public, anon;
grant execute on function public.bot_escalar(uuid, text) to authenticated;
```

> **Nota si no usas Supabase.** Las tablas son las mismas en cualquier base SQL. Lo que cambia: `auth.uid()` es el usuario de la petición en tu backend; «RLS sin políticas» significa que solo tu servidor toca estas tablas; «security invoker» significa que las consultas del contexto y de las herramientas llevan siempre el filtro por ese usuario.

Anexo D · El código de referencia

TypeScript para una función de Supabase (Deno). Es la esencia de `cerebro.ts`, generalizada a varios proveedores. Adáptala a las convenciones del proyecto.

La cadena de modelos.

TYPESCRIPT

```
// Los modelos van en un secreto: los proveedores los retiran sin avisar.
export const CADENA = (Deno.env.get("ASISTENTE_MODELOS") || "")
  .split(",").map((s) => s.trim()).filter(Boolean);

// Cada proveedor compatible con la API de chat de OpenAI: URL, secreto, prefijo.
const PROVEEDORES: Record<string, { url: string; secreto: string; prefijo?: string }> = {
  groq:       { url: "https://api.groq.com/openai/v1/chat/completions", secreto: "GROQ_API_KEY",
prefijo: "gsk_" },
  openai:     { url: "https://api.openai.com/v1/chat/completions", secreto: "OPENAI_API_KEY", prefijo:
"sk-" },
  openrouter: { url: "https://openrouter.ai/api/v1/chat/completions", secreto: "OPENROUTER_API_KEY",
prefijo: "sk-or-" },
};

// Poco razonamiento, con la palabra de cada familia; al que no razona, nada.
export function esfuerzo(modelo: string): Record<string, string> {
```

```
  if (/gpt-oss|o\d|gpt-5/i.test(modelo)) return { reasoning_effort: "low" };
  if (/qwen/i.test(modelo)) return { reasoning_effort: "none" };
  return {};
}

export async function preguntar(
  mensajes: any[],
  op: { herramientas?: any[]; tope?: number; temperatura?: number } = {},
  intento = 0, porQue: string[] = [],
  estado = { espera: 0, segundoPase: false },
): Promise<{ texto: string; llamadas: any[]; modelo: string; uso: any }> {
  if (intento >= CADENA.length) {
    // Todos dijeron 429 y alguno se libera pronto: esperar UNA vez y repetir.
    if (!estado.segundoPase && estado.espera > 0 && estado.espera <= 6000) {
      await new Promise((r) => setTimeout(r, estado.espera + 250));
      return preguntar(mensajes, op, 0, [...porQue, `se esperó ${estado.espera} ms`],
        { espera: 0, segundoPase: true });
    }
    // Nunca se acaba callando, y el error dice qué arreglar en cada eslabón.
    throw new Error("Ningún modelo respondió. " + porQue.join(" | "));
  }

  const [proveedor, modelo] = CADENA[intento].split(":");
  try {
    const p = PROVEEDORES[proveedor];
    if (!p) throw new Error(`Proveedor desconocido: ${proveedor}`);
    const clave = Deno.env.get(p.secreto);
    if (!clave) {
      // Los nombres parecidos, nunca los valores.
      const parecidos = Object.keys(Deno.env.toObject()).filter((k) => /API_KEY|KEY$/i.test(k));
      throw new Error(`Falta el secreto ${p.secreto}. Parecidos: ${parecidos.join(", ") ||
"ninguno"}.`);
    }
    if (p.prefijo && !clave.startsWith(p.prefijo)) {
      throw new Error(`${p.secreto} no empieza por ${p.prefijo}: ¿se pegó con espacios o a medias?`);
    }

    const cuerpo: Record<string, unknown> = {
      model: modelo, messages: mensajes,
      max_tokens: op.tope ?? 700, temperature: op.temperatura ?? 0.6,
      ...esfuerzo(modelo),
    };
    if (op.herramientas?.length) { cuerpo.tools = op.herramientas; cuerpo.tool_choice = "auto"; }

    const pedir = () => fetch(p.url, {
      method: "POST",
      headers: { Authorization: `Bearer ${clave}`, "Content-Type": "application/json" },
      body: JSON.stringify(cuerpo),
    });
    let res = await pedir();
    let j = await res.json().catch(() => ({}));

    // No entiende el parámetro de razonamiento: se repite sin él.
    if (res.status === 400 && cuerpo.reasoning_effort && /reasoning/i.test(JSON.stringify(j))) {
      delete cuerpo.reasoning_effort;
      res = await pedir();
      j = await res.json().catch(() => ({}));
    }
    // 429: se anota cuánto pide esperar y se pasa al siguiente YA.
    if (res.status === 429) {
      const ms = esperaQuePide(res, j);
      estado.espera = estado.espera > 0 ? Math.min(estado.espera, ms) : ms;
    }
    if (!res.ok) throw new Error(`${modelo}: HTTP ${res.status} ${JSON.stringify(j).slice(0, 200)}`);
```

```
    const m = j?.choices?.[0]?.message ?? {};
    const llamadas = m.tool_calls ?? [];
    const texto = (m.content || "").trim();
    // Vacío solo es fallo si tampoco pidió herramientas.
    if (!texto && !llamadas.length) throw new Error(`${modelo}: respondió vacío`);
    return { texto, llamadas, modelo, uso: j?.usage ?? {} };
  } catch (e) {
    console.error(`[cerebro] falló ${CADENA[intento]}: ${e}`);
    return preguntar(mensajes, op, intento + 1,
      [...porQue, `${CADENA[intento]}: ${String(e).replace(/^Error:\s*/, "")}`], estado);
  }
}

function esperaQuePide(res: Response, j: any): number {
  const cab = Number(res.headers.get("retry-after"));
  if (Number.isFinite(cab) && cab > 0) return Math.ceil(cab * 1000);
  const m = /try again in ([\d.]+)\s*(ms|s)\b/i.exec(String(j?.error?.message ?? ""));
  if (m) return Math.ceil(Number(m[1]) * (m[2].toLowerCase() === "ms" ? 1 : 1000));
  return 2000;
}
```

El bucle de herramientas.

TYPESCRIPT

```
export type Herramienta = {
  nombre: string; rpc: string; descripcion: string;
  parametros: Record<string, unknown>; obligatorios?: string[];
  delServidor?: string[];                  // los pone el servidor, el modelo no los ve
  ambito: "visitante" | "usuario" | "equipo" | "ambos"; roles?: string[];
  escribe?: boolean; entidad?: string; idEnRespuesta?: string;
};

export const enFormato = (cat: Herramienta[]) => cat.map((h) => ({
  type: "function",
  function: {
    name: h.nombre, description: h.descripcion,
    parameters: { type: "object", properties: h.parametros, required: h.obligatorios ?? [] },
  },
}));

// Con el CLIENTE DE QUIEN PREGUNTA: ahí está toda la seguridad.
export async function ejecutar(cliente: any, cat: Herramienta[], llamadas: any[],
                               delServidor: Record<string, unknown>) {
  const hechas: any[] = [];
  for (const ll of llamadas.slice(0, 4)) {
    const nombre = ll?.function?.name ?? "";
    const h = cat.find((x) => x.nombre === nombre);
    if (!h) { hechas.push({ nombre, error: "Esa herramienta no existe." }); continue; }

    let args: any = {};
    try { args = JSON.parse(ll?.function?.arguments || "{}"); }
    catch { hechas.push({ nombre, error: "No entendí los argumentos." }); continue; }

    // Solo los parámetros declarados; los de identidad, del servidor.
    const limpios: Record<string, unknown> = {};
    for (const k of Object.keys(h.parametros)) {
      if (args[k] !== undefined && args[k] !== null && args[k] !== "") limpios[k] = args[k];
    }
    for (const k of h.delServidor ?? []) if (delServidor[k] !== undefined) limpios[k] =
delServidor[k];

    try {
```

```
      const { data, error } = await cliente.rpc(h.rpc, limpios);
      if (error) { hechas.push({ nombre, error: error.message ?? String(error) }); continue; }
      hechas.push({ nombre, resultado: data });
      // Lo que escribe deja rastro, en UN solo sitio.
      if (h.escribe && h.entidad) {
        try {
          await cliente.rpc("bot_anotar_accion", {
            p_accion: h.nombre, p_entidad: h.entidad,
            p_entidad_id: h.idEnRespuesta ? data?.[h.idEnRespuesta] ?? null : null,
            p_detalle: { argumentos: limpios, resultado: data },
          });
        } catch (e) { console.error("[cerebro] no se pudo anotar:", e); }
      }
    } catch (e) {
      hechas.push({ nombre, error: String(e).slice(0, 200) });
    }
  }
  return hechas;
}

export async function conversar(o: {
  cliente: any; mensajes: any[]; catalogo: Herramienta[];
  delServidor: Record<string, unknown>; tope?: number; temperatura?: number;
}) {
  const mensajes = [...o.mensajes];
  const usadas: any[] = [];
  const herramientas = o.catalogo.length ? enFormato(o.catalogo) : undefined;
  let modelo = "", uso: any = {};

  for (let vuelta = 0; vuelta < 3; vuelta++) {
    // La última vuelta, sin herramientas: obligado a contestar.
    const r = await preguntar(mensajes, {
      herramientas: vuelta === 2 ? undefined : herramientas,
      tope: o.tope, temperatura: o.temperatura,
    });
    modelo = r.modelo;
    uso = {
      prompt_tokens: (uso.prompt_tokens ?? 0) + (r.uso?.prompt_tokens ?? 0),
      completion_tokens: (uso.completion_tokens ?? 0) + (r.uso?.completion_tokens ?? 0),
    };
    if (!r.llamadas.length) return { texto: r.texto, modelo, uso, usadas };

    mensajes.push({ role: "assistant", content: r.texto || null, tool_calls: r.llamadas });
    const hechas = await ejecutar(o.cliente, o.catalogo, r.llamadas, o.delServidor);
    usadas.push(...hechas);
    r.llamadas.forEach((ll: any, i: number) => {
      const h = hechas[i];
      if (!h) return;
      // El error se le devuelve tal cual: si se le oculta, se inventa que salió bien.
      mensajes.push({ role: "tool", tool_call_id: ll.id,
        content: JSON.stringify(h.error ? { error: h.error } : (h.resultado ?? null)) });
    });
  }
  return { texto: "", modelo, uso, usadas };
}
```

El filtro de salida.

TYPESCRIPT

```
// El guion es una preferencia; el filtro es la garantía.
export function limpiar(t: string): string {
  let x = t
```

```
    .replace(/[\u0000-\u0008\u000B\u000C\u000E-\u001F\u007F]/g, "")   // bytes de control
    .replace(/\*\*(.+?)\*\*/g, "$1").replace(/__(.+?)__/g, "$1")      // markdown
    .replace(/(^|\s)\*(\S[^*]*?)\*(?=\s|[.,;:!?)]|$)/g, "$1$2")
    .replace(/`{1,3}([^`]+)`{1,3}/g, "$1")
    .replace(/^#{1,6}\s+/gm, "")
    .replace(/[ \t]{2,}/g, " ").replace(/\n{3,}/g, "\n\n").trim();

  // Las frases que delatan al modelo se quitan ENTERAS. Lista corta a propósito.
  const DELATA = /\b(como (una? )?(IA|inteligencia artificial|modelo de lenguaje)|soy una IA|soy un
(bot|programa|asistente virtual))\b/i;
  if (DELATA.test(x)) {
    x = x.split(/(?<=[.!?…])\s+|\n+/)
      .filter((f) => f.trim() && !DELATA.test(f)).join(" ").trim();
  }
  return x;
}

// Quien llama: si el filtro lo vacía todo, gana el original.
//   texto = limpiar(r.texto) || r.texto.trim();

export function pareceTexto(t: string): boolean {
  const s = (t ?? "").trim();
  if (s.length < 8 || /[.…]{4,}/.test(s)) return false;
  const letras = (s.match(/\p{L}/gu) ?? []).length;
  return letras >= 8 && letras / s.length >= 0.5;
}
```

La red de la promesa (en la función `asistente`, después de responder).

TYPESCRIPT

```
if (conversacion && !fallo) {
  const loPrometio = /avis|notific|le paso|paso tu mensaje|te escrib|te contact/i.test(texto);
  const salioMal  = usadas.some((u) => u.nombre === "avisar_al_equipo" && u.error);
  const salioBien = usadas.some((u) => u.nombre === "avisar_al_equipo" && !u.error);
  if ((loPrometio || salioMal) && !salioBien) {
    try {
      const { error } = await clienteDeQuienPregunta.rpc("bot_escalar", {
        p_conversacion: conversacion,                  // la del servidor, no la del modelo
        p_motivo: "Lo prometió el asistente en la conversación",
      });
      if (error) throw error;
    } catch (e) {
      console.error("[asistente] no se pudo cumplir el aviso prometido:", e);
      texto = "Ahora no consigo avisar al equipo por aquí. Escríbenos por los canales "
            + "del sitio y te atienden.";            // prometer y no cumplir es peor
    }
  }
}
```

Pedir los datos una sola vez.

TYPESCRIPT

```
// Una PETICIÓN de datos: con pregunta o con verbo de pedir.
function pideDatos(s: string): boolean {
  return /nombre.{0,40}correo|correo.{0,40}nombre|tus? (datos|contacto)|un contacto|me los
(compartes|das|pasas|dejas|envias|mandas)/i.test(s)
    && /\?|dejas|d[eé]jame|dame|p[aá]same|compart|necesito|ind[ií]ca|escr[ií]beme|me das|me dices|me
pasas/i.test(s);
}
const quitarPeticion = (s: string) =>
```

```
  s.split(/(?<=[.!?])\s+/).filter((x) => !pideDatos(x)).join(" ").trim();

const yaPidioDatos = hilo.some((m) => m.role === "assistant" && pideDatos(String(m.content || "")));
// 1) el encargo cambia, 2) los guiones pierden sus líneas de contacto,
// 3) la frase se quita de la respuesta, 4) si era toda la respuesta, otra vuelta.
```

La llamada desde la ventana.

JAVASCRIPT

```
const { data, error } = await sb.functions.invoke('asistente', {
  body: {
    pregunta: texto, ambito: ambitoActual, conversacion,
    ...(ambitoActual === 'visitante' ? { huella: huellaVisitante() } : {}),
  },
});
// data: { respuesta, ambito, modelo, ms, degradado, hizo, ir, conversacion? }
```

Nota si no usas Supabase. El cerebro no depende de Supabase: es `fetch` a una API compatible con OpenAI y llamadas a funciones de datos. En otro backend, `cliente.rpc(nombre, args)` es tu función que ejecuta la herramienta **con el usuario de la petición**, y la función `asistente` es cualquier endpoint de servidor (Node, Python, PHP) que reciba el mismo cuerpo y devuelva la misma respuesta.

Anexo E · Antes de encenderlo para el público

Una lista para la persona, sin tecnicismos:

- [ ] El asistente sabe su nombre y lo dice de varias formas.
- [ ] Contesta con los precios y datos reales, y si cambias uno en el panel lo sabe en la siguiente pregunta.
- [ ] Si no hay nada a la venta, lo dice sin inventar.
- [ ] Un cliente no puede sacarle nada de otro cliente, ni insistiendo.
- [ ] Cuando alguien pide una persona, te llega el aviso a ti de verdad.
- [ ] Pide los datos de contacto una vez y no insiste.
- [ ] Si el proveedor de IA falla, contesta con una frase honesta y lo ves en el panel.
- [ ] Tienes dos proveedores de IA en la cadena.
- [ ] Las conversaciones quedan guardadas y puedes leerlas.
- [ ] Sabes dónde enseñarle un dato nuevo y dónde corregirle una respuesta.
- [ ] Los topes de gasto están puestos y sabes cuánto cuesta un mes normal.
- [ ] La IA del proyecto pasó las dieciocho pruebas del P11 y te enseñó la tabla.